

华中科技大学

课程实验报告

课程名称: C 语言程序设计实验

专业班级 CS2404

学号 U202490035

姓名 阮云轩

指导教师 卢萍

报告日期 2024 年 11 月 22 日

计算机科学与技术学院

目 录

1	摘要	1
1.1	關於行文	1
1.2	前言	1
2	实验五数组	2
2.1	5-1 程序改错与跟踪调试	2
2.2	程序完善与修改替换	9
2.3	程序设计	17
2.4	实验五小结	56
3	实验七结构与联合	60
3.1	程序改错与跟踪调试	60
3.2	程序完善与修改替换	62
3.3	程序设计	68
3.4	实验七小结	113
4	總結心得	117
5	參考文獻	118

1 摘要

1.1 關於行文

由於筆者習慣以漢字 (繁體中文) 以行文，在實驗報告中，在杜撰時經已盡量轉換成漢字 (簡體中文)，但難免會有漢字 (繁體中文) 的出現。還請讀者見諒。

1.2 前言

本 C 语言程序课程实验报告是由 LaTeX 编写的，主要写了关于数组和结构的实验题。本实验报告包括但不限于对实验题目的分析，提出对题目的理解和需要注意的相关事项，构建对应的设计思路，利用相关流程图去说明以便于和个人读者理解，给出相关报行代码供以读者自运行测试，还带有章节的小结点出相关的数据结构和编程思维，最后还有个人感悟指点对 C 语言的理解和个人收获。由于作者水平有限，难免有些缺点和错误，望广大读者给予批评指正。

Listing 1 示例

```
1      #include <stdio.h>
2      int main() {
3          printf("Hello, C!\n"); //print out Hello, C!
4          return 0;
5      }
```

以上是本實驗報告時頻繁出現的編程文本格式，示味着相關的說明部分 (1)

2 实验五数组

2.1 5-1 程序改错与跟踪调试

目标要求:

在下面所给的源程序中, 函数 `strncate(t,s)` 的功能是将字符串 `s` 连接到字符串 `t` 的尾部; 函数 `strdelc(s,c)` 的功能是从字符串 `s` 中删除所有与给定字符 `c` 相同的字符, 程序应该能够输出如下结果 (2):

Listing 2 示例

```
1      Programming Language
2      ProgrammingLanguageLanguage
3      ProgrmmingLnguage
```

跟踪和分析源程序中存在的问题, 排除程序中的各种逻辑错误, 使之能够输出正确结果。

2.1.1 相关修正补充

1) 目标程序

Listing 3 目标程序

```
1      /*实验5-1程序改错与跟踪调试题程序*/
2      #include<stdio.h>
3      void strncate (char [], char []);
4      void strdelc (char [], char);
5      int main (void)
6      {
7          char a[ ]="Language", b[ ]="Programming"; //问题一
8          printf("%s %s\n",b,a);
9          strncate (b, a); printf("%s %s\n",b,a);
10         strdelc(b, 'a'); printf("%s\n",b);
11         return 0;
12     }
13     void strncate(char t[ ], char s[])
14     {
15         int i=0,j=0;
16         while(t[i++]); //问题二
```

```
17         while((t[i++] = s[j++])!='\0');
18     }
19     void strdelc (char s[ ], char c)
20     {
21         int j, k;
22         for(j=k=0; s[j]!='\0' ; j++)
23             if(s[j] !=c) s[k++]=s[j]; //问题三
24     }
```

对上面的代码 (3)，对程序执行有影响的问题主要有两处，除此之外，编写时亦应要注意代码的可读性，尽量简洁清晰易于理解，此点会呈现在之后修改后的代码中并一同解决，若后续实验题存在类似情况亦会对程序代码进行编排故不作赘述。

2) 程序改错

一、char a[]="Language", b[]="Programming";

分析：

这里算是一个硬性问题，并不是代码逻辑的问题，我在除错时也疑惑了不少，出现的现象是在 strcat 函数处理完 char a 和 char b 后再输出时发现 char a 的内容被改变了，但这十分不合理因为在代码中没有对 char a 内容赋值修改的操作。

而原因是这个 strcat 函数将字符串 s 追加到字符串 t 的末尾。然而，如果在追加过程中，t 的长度不足以容纳 s 的所有字符，那么 t 的内容可能会被截断，导致 a 的内容被修改。为了确认这一点，我们需要检查 a 和 b 的长度，以及 strcat 函数在追加过程中是否发生了溢出。如果 b 的长度不足以容纳 a 的所有字符，那么 a 的内容可能会被修改。

处理方法：

1. 确定 char b 字符串的容量4

Listing 4 示例

```
1     char b[10086];
```

描述：定义这个字符串容量的前提是要确定这个字符串的容量始终有上限。但同时无法完全利用，存在无用的内存空间。

优点：简单快捷

缺点：无法动态分配，对函数的内存空间无法动态优化

2. 利用 malloc, realloc 动态分配5

Listing 5 示例

```
1      int *a;  
2      a = (int *)malloc(100 * sizeof(int));  
3      a = (int *)realloc(a, 200 * sizeof(int));
```

描述：利用了 malloc 动态分配，实现对代码效率的提升。

优点：动态利用，可随时分配空间

二、strcat 函数的处理

分析：

strcat 是一个把字符串连接在一起的函数，但注意子串的首元素下标是“0”开始，故对于函数内的“i++”计数实际上是计字符串的长度，但使用时其实下标会比原来的少一。因此要对 i 计数进行编辑，另外，这种的计数方法是依赖于数据内容是否为 NULL，当对字符串内容进行改变时难免会出现数据残留问题

由于 strcpy 不会自动清零目标字符串中剩余的部分，所以如果之前目标字符串中有一些旧的数据，这些数据可能会残留在字符串后面的部分。这可能会在后续处理字符串（比如比较字符串长度、进行字符串拼接等操作）时产生意外的结果。

处理方法：

计数处理：

1. 对计数变量人为控制

简单地把计数变量“i”减 1，使其刚好指向“\0”结束符的位置

数据残留处理：

1. 人为控制

在字符串结束的后一位人为加一个“\0”结束符。

如 (6)：

Listing 6 示例

```
1      a[i++] = '\0'
```

2. 使用 memset 函数

在进行更改前先进行一次清零

如 (7)：

Listing 7 示例

```
1      void *memset(void *s, int c, size)
```

三、strdelc 函数的处理

分析:

strdelc 函数的目的是从给定的字符串 s 中删除指定的字符 c。其基本思路是通过一个循环遍历字符串 s，当字符不等于 c 时，将该字符复制到新的位置（由索引 k 控制），最后在合适的位置添加字符串结束符 '\0' 以保证字符串的完整性。

处理方法:

1. 在对删减后的字符串尾部添加结束符标示

如 (8):

Listing 8 示例

```
1 标识符 [i] = '\0'
```

综合以上可见问题，对代码进行修改，以下 (9) 是修改后的代码，将会对此代码进行跟踪调试以检查是否顺利执行及对代码运行过程以理解。

Listing 9 示例

```
1  #include<stdio.h>
2  void strcate(char [], char[]);
3  void strdelc(char [], char);
4  int main()
5  {
6      char a[] = "Language", b[1000] = "Programming";
7      printf("%s %s\n", b, a);
8      strcate(b, a);
9      printf("%s %s\n", b, a);
10     strdelc(b, 'a');
11     printf("%s\n", b);
12     return 0;
13 }
14 void strcate(char t[], char s[])
15 {
16     int i = 0, j = 0;
17     while (t[i] != '\0') {
18         i++;
19     }
20     while (s[j] != '\0') {
21         t[i++] = s[j++];
22     }
```

```
23         t[i] = '\0';
24     }
25     void strdelc(char s[], char c)
26     {
27         int j, k;
28         for (j = k = 0; s[j] != '\0'; j++)
29         {
30             if (s[j] != c)
31                 s[k++] = s[j];
32         }
33         s[k] = '\0';
34     }
```

3) 跟踪调试

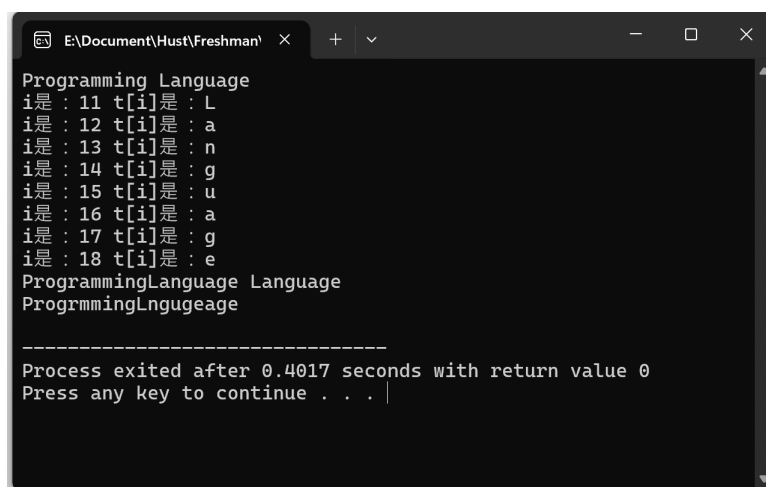
(1) 单步执行源程序。跟踪进入函数 `strcate` 时, 观察字符数组 `t` 和 `s` 中的内容, 分析结果是否正确。当单步执行光条刚落在第二个 `while` 语句所在行时, `i` 为何值? `t[i]` 为何值? 分析该结果是否存在的问题。当单步执行光条落在 `strcate` 函数块结束标记即右花括号”)”所在行时, 字符数组 `t` 和 `s` 分别为何值? 分析是否实现了字符串连接。

思路:

这里只要简单加一个 `printf` 命令 (10) 就可以检视当前的 `i` 和 `t[i]`, 但注意因为简洁书写时已进行自增操作, 因此要检视当前的数据需要进行「减 1」的操作 (2-1)

Listing 10 示例

```
1 printf("i是: %d t[i]是: %c \n", i-1, t[i-1]);
```



```
Programming Language
i是: 11 t[i]是: L
i是: 12 t[i]是: a
i是: 13 t[i]是: n
i是: 14 t[i]是: g
i是: 15 t[i]是: u
i是: 16 t[i]是: a
i是: 17 t[i]是: g
i是: 18 t[i]是: e
ProgrammingLanguage Language
ProgrmmingLngeage

-----
Process exited after 0.4017 seconds with return value 0
Press any key to continue . . .
```

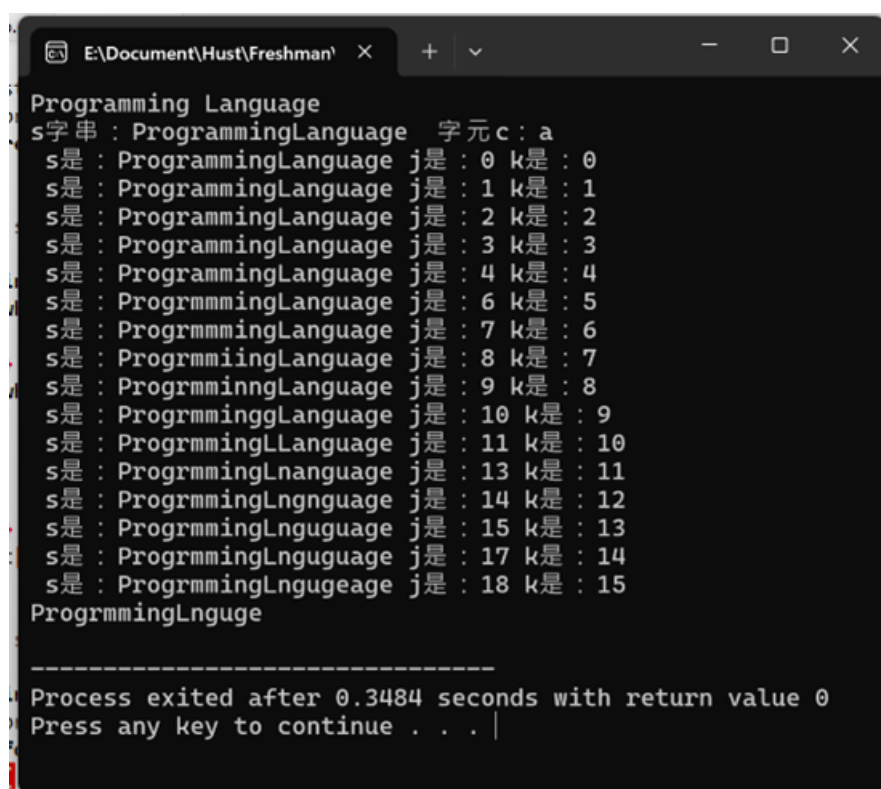
图 2-1 执行示例

如结果所示, 程序成功地把「Language」拼接到字符串后面, 完善了 `strcate` 的功能

(2) 跟踪进入函数 `strdelc` 时, 观察字符数组 `s` 中的内容和字符 `c` 的值, 分析结果是否正确。单步执行 `for` 语句过程中, 观察字符数组 `s`、`j` 和 `k` 值的变化, 分析该结果是否存在问题。当单步执行光条落在 `strdele` 函数块结束标记所在行时, 字符串 `s` 为何值? 分析是否实现了所要求的删除操作。

思路:

我們利用迴圈把每次的狀態輸出即可 (2-2)。



```
Programming Language
s字串: ProgrammingLanguage 字元c: a
s是: ProgrammingLanguage j是: 0 k是: 0
s是: ProgrammingLanguage j是: 1 k是: 1
s是: ProgrammingLanguage j是: 2 k是: 2
s是: ProgrammingLanguage j是: 3 k是: 3
s是: ProgrammingLanguage j是: 4 k是: 4
s是: ProgrmmingLanguage j是: 6 k是: 5
s是: ProgrmmingLanguage j是: 7 k是: 6
s是: ProgrmimingLanguage j是: 8 k是: 7
s是: ProgrmminngLanguage j是: 9 k是: 8
s是: ProgrmminggLanguage j是: 10 k是: 9
s是: ProgrmmingLLanguage j是: 11 k是: 10
s是: ProgrmmingLnanguage j是: 13 k是: 11
s是: ProgrmmingLngnuguage j是: 14 k是: 12
s是: ProgrmmingLnguguage j是: 15 k是: 13
s是: ProgrmmingLnguguage j是: 17 k是: 14
s是: ProgrmmingLngugeage j是: 18 k是: 15
ProgrmmingLnguge

-----
Process exited after 0.3484 seconds with return value 0
Press any key to continue . . . |
```

图 2-2 執行示例

这里 (2-2) 虽然看上去似乎有点乱, 但其实就是不断在替换字符, 当遇到目标字符时就跳过字符, 这时的 `k` 也不会自增, 设计这两个变量 `J`、`k`, `j` 负责遍历字符串, `k` 负责对字符串实现替换操作。

总的来说, 在对程序进行修整后, 跟踪调试出来的结果亦是正常的。注意到目前处理的数据都是自定义常量, 要使这个程序足以成为一个「工具」还需加入客户端输入的功能, 亦即是头歌设计的要求, 要满足此条件, 在原有的程序上加入「`scanf`」输入命令即可。以下为头歌要求的代码 (11):

Listing 11 示例

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  void strcat(char t[], char s[]);
5  void strdelc(char s[], char c);
6  int main()
7  {
8      char a[100], b[100];
9      char c;
10     scanf("%s %s %c", b, a, &c);
11     strcat(b, a);
12     printf("%s\n", b);
13     strdelc(b, c);
14     printf("%s\n", b);
15     return 0;
16 }
17 void strcat(char t[], char s[])
18 {
19     int i = 0, j = 0;
20     while (t[i] != '\0') {
21         i++;
22     }
23     while (s[j] != '\0') {
24         t[i++] = s[j++];
25     }
26     t[i] = '\0';
27 }
28 void strdelc(char s[], char c)
29 {
30     int j, k;
31     for (j = k = 0; s[j] != '\0'; j++) {
32         if (s[j] != c) {
33             s[k++] = s[j];
34         }
35     }
36     s[k] = '\0';
37 }
```

2.2 程序完善与修改替换

2.2.1 5-2 去掉重复字符

1) 目标内容

(1) 在一些文本数据处理中, 希望去掉重复 (简称去重) 的字符, 仅保留首次出现的字符。下面的程序能够删除字符串中的重复字符。例如源串为 12eerer, 去重后为 12er。程序采用二重循环, 大循环每次从字符串中取出一个字符, 小循环遍历字符串中该字符后面的所有字符, 若有重复的字符, 则将后面的字符设置为空字符\0。请在源程序中的下画线处填写合适的代码来完善该程序 (13)。

Listing 12 示例

```
1      #include<stdio.h>
2      #include<string.h>
3      void RemoveDuplicate(char *s);
4      int main()
5      {
6          char str[200];
7          while(fgets(str, 200, stdin) != NULL)
8          {
9              RemoveDuplicate(str);
10             printf("%s", str);
11         }
12         return 0;
13     }
14
15     void RemoveDuplicate(char *s)
16     {
17         int r, w, i, len;
18         len = strlen(s);
19         for (r = w = 0; r < len; r++)
20         {
21             if (s[r] != '\0')
22             {
23                 s[w] = s[r];
24                 w++;
25                 for (i = r + 1; i < len; i++)
26                 {
27                     if (s[i] == s[r])
```

```
28         {
29             s[i] = '\0';
30         }
31     }
32 }
33 }
34 s[w] = '\0';
35 }
```

執行結果 (2-3):

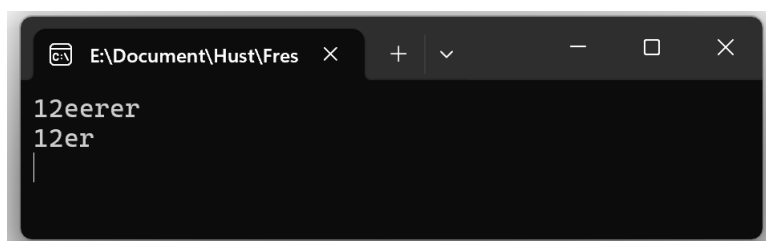


图 2-3 執行示例

说明:

可见这里的代码是没有问题的,能够顺利完成去重的操作。这里代码的设计思路和刚才那题相似,设计了二重循环,外部循环负责遍历字符串,内部循环负责判断能否去重,但同时内部循环的设计同时使用遍历去设计,使得程序的时间复杂度大大增加,面对大量数据时尤为明显。

2.2.2 5-3 去掉重复字符

1) 目标内容

(2) 上面程序的时间复杂度为 $O(n^2)$ 。可以采用空间换时间的方法,提高处理的时间效率。具体方法为: 假设待处理文本都为 ASCII 码字符 (共 256 个), 设置一个大小为 256 的标记数组, 记录每个字符是否已出现过, 每遇到一个字符, 根据该字符的 ASCII 码值, 将数组对应元素置为 1。所以, 判断一个字符是否重复, 只需检查标记数组中相应位置上的值。请采用标记数组法修改 RemoveDuplicate 函数, 使用一重循环实现字符去重, 使其时间复杂度为 $O(n)$

Listing 13 示例

```
1 #include <stdio.h>
2 #include <string.h>
```

```
3      void RemoveDuplicate(char *s);
4      int main()
5      {
6          char str[200];
7          while (fgets(str, 200, stdin) != NULL)
8          {
9              RemoveDuplicate(str);
10             printf("%s", str);
11         }
12         return 0;
13     }
14     void RemoveDuplicate(char *s)
15     {
16         int i, len;
17         len = strlen(s);
18         int mark[256] = {0};
19         int w = 0;
20         for (i = 0; i < len; i++)
21         {
22             int ascii_index = (int)s[i];
23             if (mark[ascii_index] == 0)
24             {
25                 mark[ascii_index] = 1;
26                 s[w++] = s[i];
27             }
28         }
29         s[w] = '\0';
30     }
```

执行结果 (2-4):

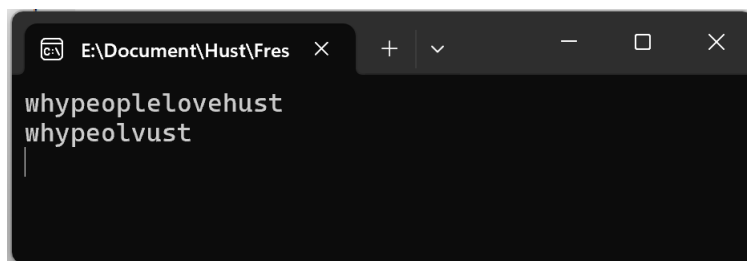


图 2-4 执行示例

这个设计同样是能够顺利去重的。

这里按照替代方案的思路设计了一个判断被存入字符是否已被输入过的数组，这

个数组主要以放入 0/1 布尔值去作为条件句执行。同样能够实现去重的操作，但相比起原来的内部循环更简化了时间复杂度。

2.2.3 5-4 约瑟夫问题 (一)

1) 目标内容

(1) 下面的源程序用于求解约瑟夫问题:M 个人围成一圈, 从第一个人开始依次从 1 至 N 循环报数, 每当报数为 N 时报数人出圈, 直到圈中只剩下一个人为止。

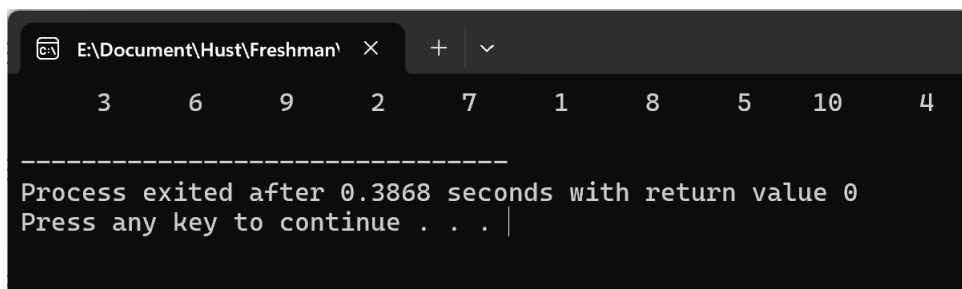
相关代码 (14):

Listing 14 示例

```
1      #include <stdio.h>
2
3      #define M 10
4      #define N 3
5      int main(void)
6      {
7          int a[M], b[M];
8          int i, j, k;
9          for (i = 0; i < M; i++)
10         {
11             a[i] = i + 1;
12         }
13         for (i = M, j = 0; i > 1; i--)
14         {
15             for (k = 1; k <= N; k++)
16             {
17                 if (++j > i - 1)
18                     j = 0;
19             }
20             b[M - i] = j? a[j - 1] : a[i - 1];
21             if (j)
22             {
23                 for (k = --j; k < i - 1; k++)
24                 {
25                     a[k] = a[k + 1];
26                 }
27             }
28         }
29         else
```

```
28         {
29             for (k = i - 1; k > 0; k--)
30             {
31                 a[k] = a[k - 1];
32             }
33         }
34     }
35     for (i = 0; i < M - 1; i++)
36     {
37         printf("%6d", b[i]);
38     }
39     printf("%6d\n", a[0]);
40     return 0;
41 }
```

输出结果:



```
E:\Document\Hust\Freshman>
3      6      9      2      7      1      8      5      10      4
-----
Process exited after 0.3868 seconds with return value 0
Press any key to continue . . .
```

图 2-5 执行示例

这里 (2-5) 定义了「M=10」、「N=3」去进行约瑟夫问题，从输出结果来看这段代码是正确的，能够顺利执行。

这段代码中，外层的 for 循环 `for (i = M; i > 1; i--)` 控制整个出圈的过程，i 初始化为 M 表示初始圈里有 M 个人，随着每次有人出圈，i 的值会减 1，直到圈里只剩下 1 个人（即 `i > 1` 这个条件不满足时循环结束），j 用于记录当前报数的人的位置（下标），初始化为 0。

中层的 for 循环 `for (k = 1; k <= N; k++)` 用于模拟每次从 1 到 N 的报数过程。在这个循环里，`if (++j > i - 1)` 这个条件判断实现了循环报数的逻辑，当 j 的值（经过自增后）超过了当前圈里剩下的人数减 1（因为数组下标从 0 开始）时，就把 j 重置为 0，相当于报数到一圈的末尾后又回到开头继续报数。

2.2.4 5-5 约瑟夫问题 (二)

1) 目标内容

(2) 上面的程序使用数组元素的值表示圈中人的编号, 故每当有人出圈时都要压缩数组, 这种算法不够精练。如果采用做标记的办法, 即每当有人出圈时对相应数组元素做标, 从而可省掉压缩数组的时间, 这样处理效率会更高, 请采用做标记的办法修改程序、使修改后的程序与原程序具有相同的功能。

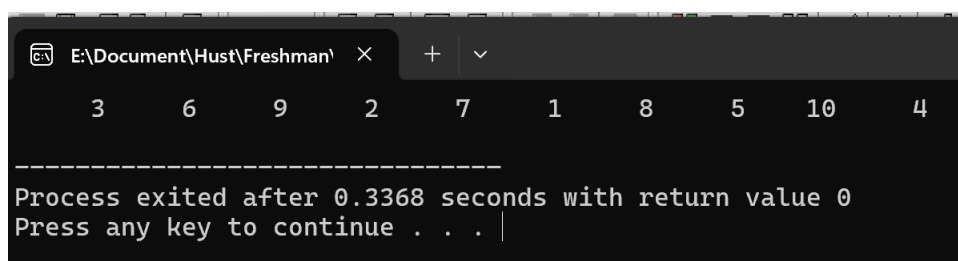
相关代码 (15):

Listing 15 示例

```
1      #include <stdio.h>
2
3      #define M 10
4      #define N 3
5      int main(void)
6      {
7          int a[M];
8          int flag[M];
9          int i, j, k;
10         for (i = 0; i < M; i++)
11         {
12             a[i] = i + 1;
13             flag[i] = 0;
14         }
15         j = -1;
16         for (i = M; i > 1; i--)
17         {
18             for (k = 1; k <= N; k++)
19             {
20                 do
21                 {
22                     j = (j + 1) % M;
23                 } while (flag[j]);
24                 flag[j] = 1;
25                 printf("%6d", a[j]);
26             }
27             for (i = 0; i < M; i++)
28             {
29                 if (flag[i] == 0)
```

```
30         {
31             printf("%6d\n", a[i]);
32             break;
33         }
34     }
35     return 0;
36 }
```

输出结果:



```
E:\Document\Hust\Freshman  ×  +  v
3      6      9      2      7      1      8      5      10      4
-----
Process exited after 0.3368 seconds with return value 0
Press any key to continue . . . |
```

图 2-6 执行示例

分析:

在这段代码 (2-6) 中, 通过定义两个数组 `a` 和 `flag` 以及相应的循环逻辑来实现这个过程。和上一题不同的是, 数组 `flag` 用于标记每个人是否出圈, 初始时通过循环将 `flag` 数组的所有元素都赋值为 0, 表示所有人都还未出圈。

而 `while (flag[j])` 这个条件是为了跳过那些已经被标记出圈 (对应 `flag` 数组中元素值为 1) 的位置, 保证报数是针对未出圈的人进行的。而这里把「`j=0`」改成「`j = (j + 1)`」

相关代码 (16):

Listing 16 示例

```
1      #include <stdio.h>
2      int main(void)
3      {
4          int M,N;
5          scanf("%d %d",&M,&N);
6          int a[M];
7          int flag[M];
8          int i, j, k;
9          for (i = 0; i < M; i++)
10         {
```

```
11         a[i] = i + 1;
12         flag[i] = 0;
13     }
14
15     j = -1;
16     for (i = M; i > 1; i--)
17     {
18         for (k = 1; k <= N; k++)
19         {
20             do
21             {
22                 j = (j + 1) % M;
23             } while (flag[j]);
24         }
25         flag[j] = 1;
26         printf("%6d", a[j]);
27     }
28     for (i = 0; i < M; i++)
29     {
30         if (flag[i] == 0)
31         {
32             printf("%d\n", a[i]);
33             break;
34         }
35     }
36     return 0;
37 }
```

2.3 程序设计

2.3.1 5-6 整数的二进制表示

1) 目标内容

输入一个整数，将它在内存中的二进制表示的每一位转换成对应的数字字符并且存放到一个字符数组中，然后输出该整数的二进制表示（32 位有符号整数，输出每 4 位用空格分隔）。

2) 分析

要实现输入一个整数，将其转换为 32 位有符号整数的二进制表示（每 4 位用空格分隔）并输出的功能，整体思路是利用位运算按位提取该整数的二进制位信息，转换为字符后存入数组，再按格式要求输出数组内容。

思路：

1. 获取一个用户端的整数型数据。

2. 转换过程

使用按位操作逐位检查整数的每一位。将每一位的结果存储在一个字符数组单元中。

可用三目运算符快速判断和赋值

3. 格式化输出

在每 4 位之间添加一个空格。

用取模即可；如「`iprintf`」打印最终的二进制字符串

流程图：

这里主要设计一个总流程图 (2-7) 和按位操作及存取时的流程图 (2-8) 以演示程序的运行。

首先是总流程，本题的总流程图较为单一，为一个单向无分支的操作。固能够简单容易地理解，大致上和上面思路的形容差不多，先获取数据，然后按位操作判断，最后作输出

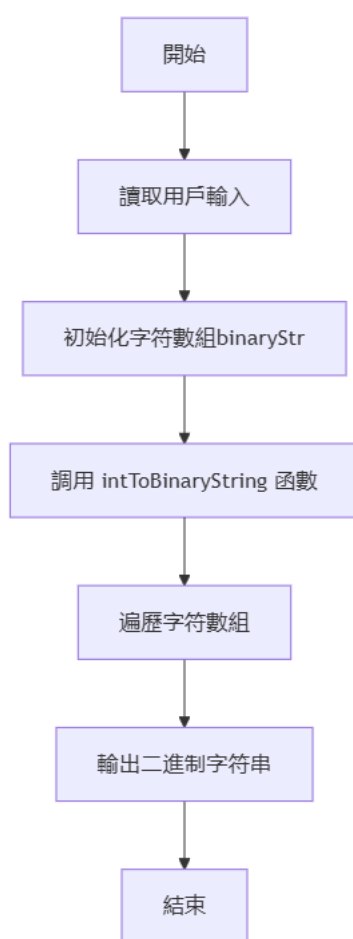


图 2-7 流程图

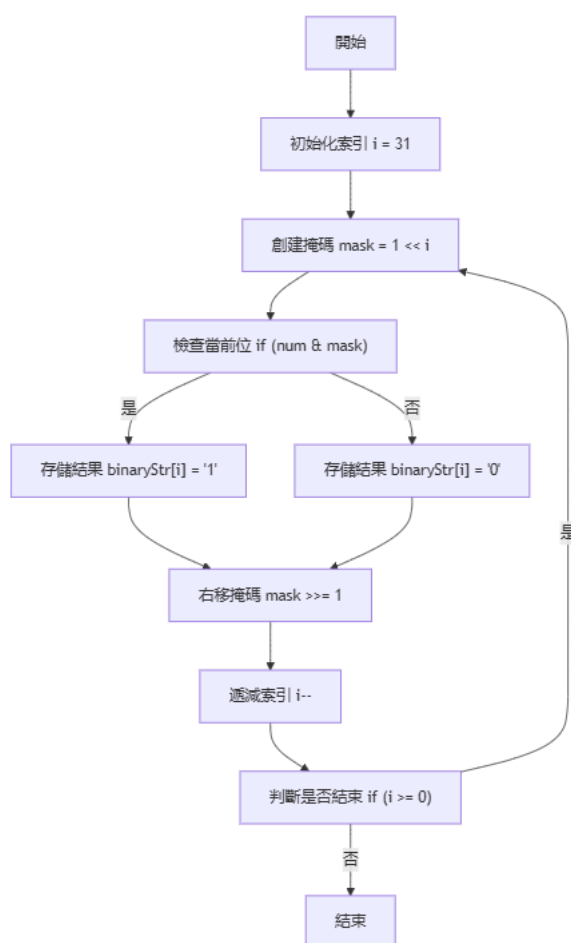


图 2-8 流程图

3) 相关代码 (17)

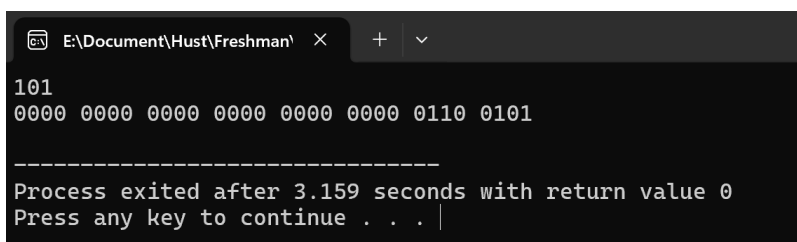
Listing 17 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #define BITS 32
4      void intToBinaryString(int num, char binaryStr[]) {
5          int i;
6          for (i = BITS - 1; i >= 0; i--) {
7              binaryStr[i] = (num & (1 << (BITS - 1 - i)))? '1' :
                        '0';
8          }
9          binaryStr[BITS] = '\0';
10     }
11     int main() {
12         int num;
```

```
13         char binaryStr[BITS + 1];
14         scanf("%d", &num);
15         intToBinaryString(num, binaryStr);
16         for (int i = 0; i < BITS; i++) {
17             printf("%c", binaryStr[i]);
18             if ((i + 1) % 4 == 0 && i != BITS - 1) {
19                 printf(" ");
20             }
21         }
22         printf("\n");
23         return 0;
24     }
```

4) 执行结果

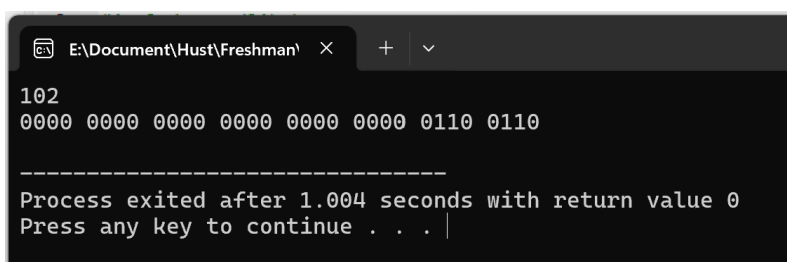
执行结果 (2-92-10)



```
E:\Document\Hust\Freshman' x + v
101
0000 0000 0000 0000 0000 0000 0110 0101

-----
Process exited after 3.159 seconds with return value 0
Press any key to continue . . . |
```

图 2-9 执行示例



```
E:\Document\Hust\Freshman' x + v
102
0000 0000 0000 0000 0000 0000 0110 0110

-----
Process exited after 1.004 seconds with return value 0
Press any key to continue . . . |
```

图 2-10 执行示例

2.3.2 5-7 成绩排序

1) 目标内容

(1) 输入一个整数, 将它在内存中二进制表示的每一位转换成对应的数字字符并且存放到一个字符数组中, 然后输出该整数的二进制表示。

(2) 编写一个 C 程序, 要求采用模块化程序设计思想, 将相关功能用函数实现, 并提供菜单选项。该程序具有以下功能:

1. 输入 n 个学生的姓名和 C 语言课程的成绩。
2. 将成绩按从高到低的次序排序, 姓名同时进行相应调整。
3. 输出所有学生的姓名和 C 语言课程的成绩。

2) 分析

整个程序可以用模块化设计思想, 将不同的功能封装到不同的函数中, 通过一个菜单来让用户选择具体要执行的功能, 这样使得程序结构清晰、易于维护和扩展。主函数主要负责展示菜单以及根据用户选择调用相应的功能函数来完成操作。

总流程图 (2-11):

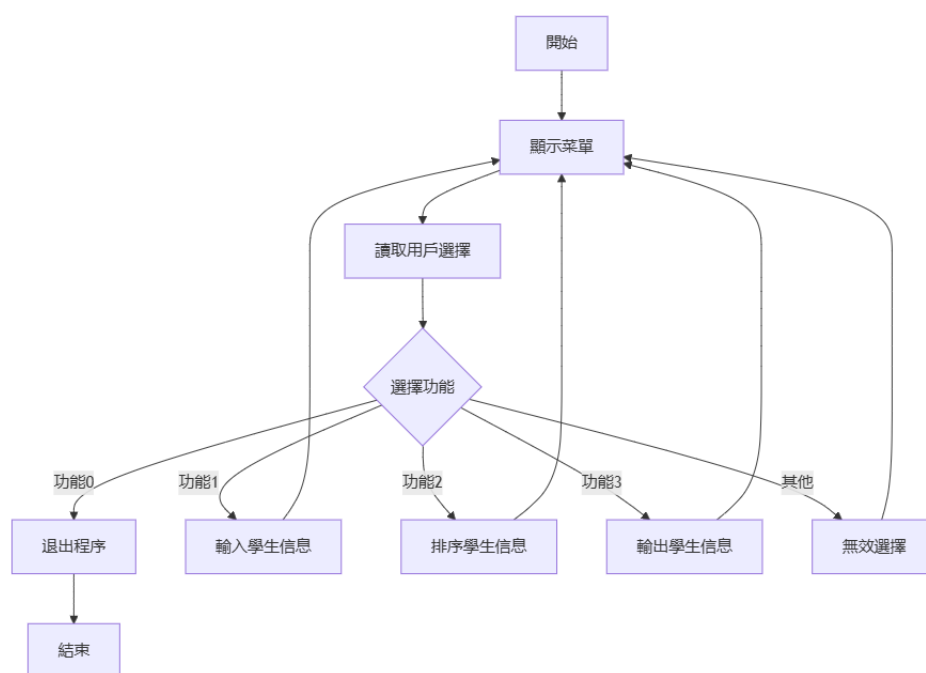


图 2-11 流程图

設計思路：

1. 输入学生信息功能实现思路：

函数设计：创建 void 函数，用于输入 n 个学生的姓名和 C 语言课程成绩。函数可以接收学生数量 n 作为参数，在函数内部定义合适的数组（比如字符数组存储姓名，整型数组存储成绩）来存放数据，通过循环让用户依次输入每个学生的姓名和成绩，完成信息的收集工作。

相关流程图 (2-12)：

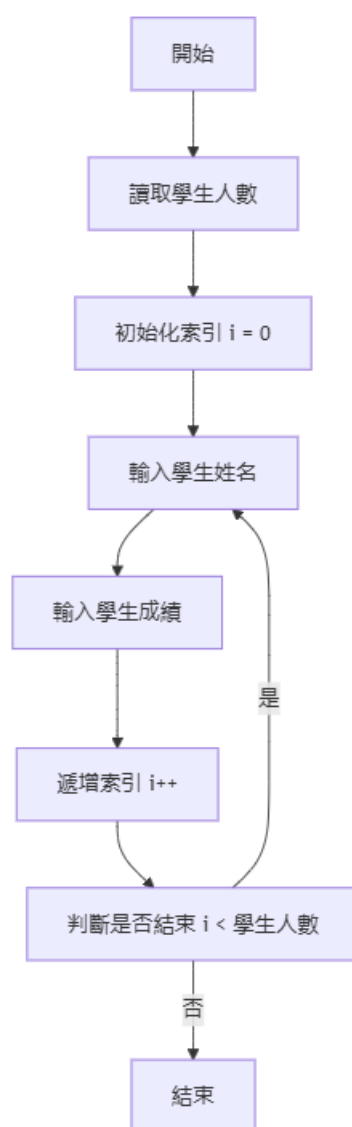


图 2-12 流程图

2. 成绩排序及姓名相应调整功能实现思路：

函数设计：设计 `void` 函数，该函数接收存储成绩和姓名的数组以及学生数量 `n` 作为参数，其目的是根据成绩对学生信息进行排序。可以采用常见的排序算法，比如冒泡排序、快速排序等，在比较成绩的同时，当交换成绩的顺序时，对应的学生姓名也要进行交换，以保证成绩和姓名始终是对应的，最终实现成绩从高到低排序且姓名同步调整的效果。

相关流程图 (2-13)：

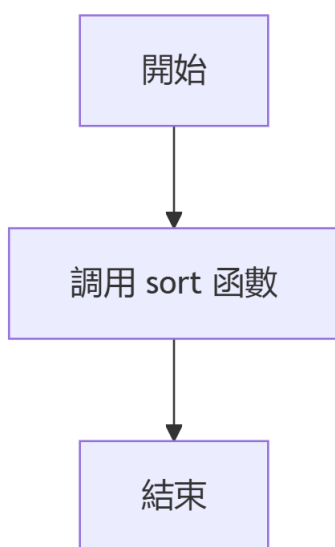


图 2-13 流程图

3. 输出学生信息功能实现思路:

函数设计: 构建 void 函数, 它接收存储学生信息的数组以及学生数量 n 作为参数, 通过循环遍历数组, 将每个学生的姓名和成绩按照一定格式输出, 方便查看所有学生的相关信息。

相关流程图 (2-14):

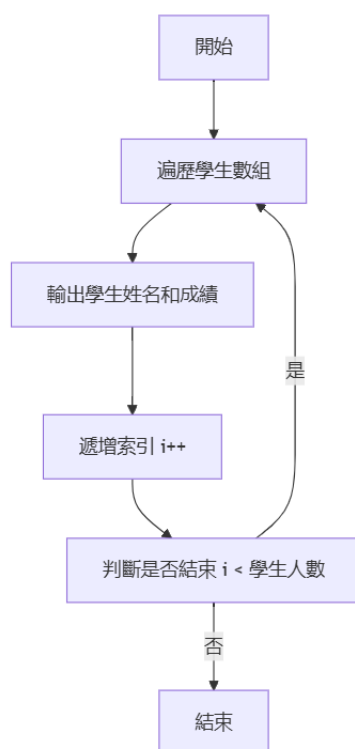


图 2-14 流程图

3) 相关代码

相关代码 (18):

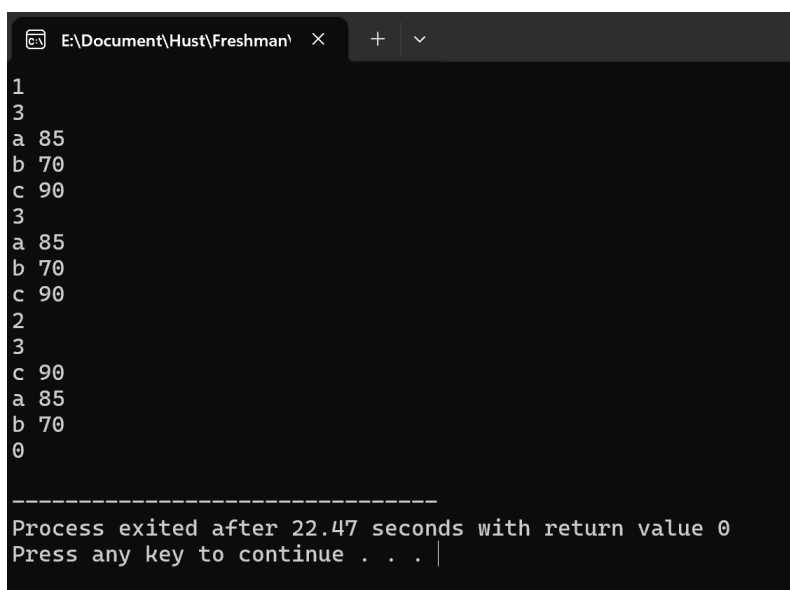
Listing 18 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4      #define MAX_STUDENTS 100
5      #define MAX_NAME_LENGTH 50
6      typedef struct {
7          char name[MAX_NAME_LENGTH];
8          int score;
9      } Student;
10     void inputStudents(Student students[], int n);
11     void sortStudents(Student students[], int n);
12     void outputStudents(Student students[], int n);
13     int main() {
14         Student students[MAX_STUDENTS];
15         int choice, n;
16         do {
```

```
17         scanf("%d", &choice);
18
19         switch (choice) {
20             case 1:
21                 scanf("%d", &n);
22                 inputStudents(students, n);
23                 break;
24             case 2:
25                 sortStudents(students, n);
26                 break;
27             case 3:
28                 outputStudents(students, n);
29                 break;
30             case 0:
31                 break;
32         }
33     } while (choice!= 0);
34     return 0;
35 }
36 void inputStudents(Student students[], int n) {
37     for (int i = 0; i < n; i++) {
38
39         scanf("%s %d", students[i].name, &students[i].score);
40     }
41 }
42 void sortStudents(Student students[], int n) {
43     for (int i = 0; i < n - 1; i++) {
44         for (int j = i + 1; j < n; j++) {
45             if (students[i].score < students[j].score) {
46                 int tempScore = students[i].score;
47                 students[i].score = students[j].score;
48                 students[j].score = tempScore;
49                 char tempName[MAX_NAME_LENGTH];
50                 strcpy(tempName, students[i].name);
51                 strcpy(students[i].name, students[j].name);
52                 strcpy(students[j].name, tempName);
53             } else if (students[i].score == students[j].score) {
54                 if (strcmp(students[i].name,
55                             students[j].name) > 0) {
56
57                     char tempName[MAX_NAME_LENGTH];
```

```
57         strcpy(tempName, students[i].name);
58         strcpy(students[i].name,
59                 students[j].name);
60         strcpy(students[j].name, tempName);
61         int tempScore = students[i].score;
62         students[i].score = students[j].score;
63         students[j].score = tempScore;
64     }
65 }
66 }
67 }
68 void outputStudents(Student students[], int n) {
69     for (int i = 0; i < n; i++) {
70         printf("%s %d\n", students[i].name, students[i].score);
71     }
72 }
```

4) 执行结果



```
E:\Document\Hust\Freshman' x + v
1
3
a 85
b 70
c 90
3
a 85
b 70
c 90
2
3
c 90
a 85
b 70
0
-----
Process exited after 22.47 seconds with return value 0
Press any key to continue . . .
```

图 2-15 执行示例

分析:

可以见到先是进行了功能 1，输入了 3 名学生的姓名和成绩，然后通过 3 直接输出，再测试排序的功后输入了 2 之后再进行输出，可见是成功排序的。
这里的代码大致可分为四个模块去观察，主函数及三功能函数部分，首先是利用

switch 语句去实现分块处理，定义了一个 structure 结构去存放学生姓名和成绩。通过 for 循环依次存入。然后在排序模块中通过创建 temp 变量和 if 条件句去进行交换内容和排序，最后输出部分则是简单的回环输出。

由此可见，把复杂的程序要求对其分块处理可以使得程序思路变得清晰，亦能更于理解读写，在除错了亦能更快定位到错处，因此养成一个模块化的思想方向可以有助于我们编程的思路。

2.3.3 5-8 二分查找

1) 目标内容

在 5-7 的基础上增加查找功能（功能编号 4）：输入一个 C 语言课程成绩值，用二分查找进行搜索。如果查找到有该成绩，则输出该成绩所有学生的姓名和 C 语言课程的成绩；否则，输出提示“not found”。

2) 分析

在原有主程序 switch 部分加入一个 case 4，然后构建二分查找法相关的函数。在使用二分法查找时要先对成绩进行排序，排序后再查找。总流程图 (2-16)：

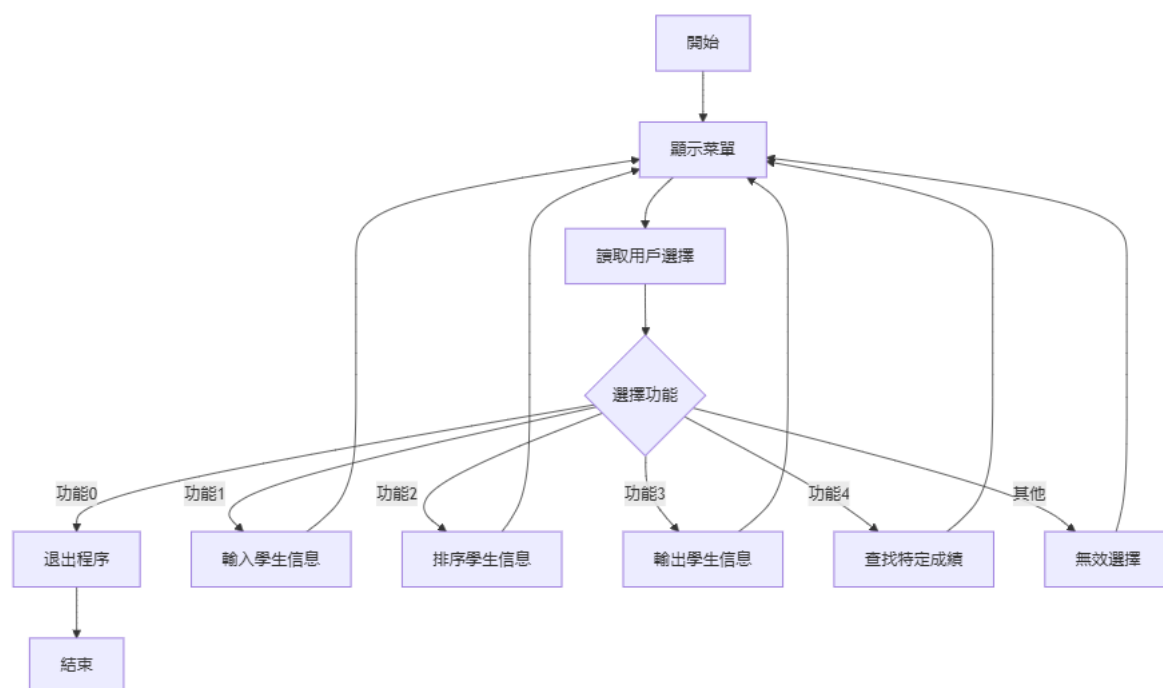


图 2-16 流程圖

这里使用 while 循环来实现二分查找的核心逻辑，只要 left 小于等于 right，就意味着还有可能在当前界定的范围内找到目标成绩，循环会持续进行。在每次循环中，首先通过 $\text{int mid} = \text{left} + (\text{right} - \text{left}) / 2$; 计算中间位置 mid。这种计算方式可以避免直接使用 $(\text{left} + \text{right}) / 2$ 时可能出现的整数溢出问题（当 left 和 right 的数值较大时），确保能正确获取中间索引。然后通过 if-else if 条件语句来根据中间位置元素（students[mid].score）与目标成绩 target 的比较结果决定下一步操作：

相关流程图 (2-17):

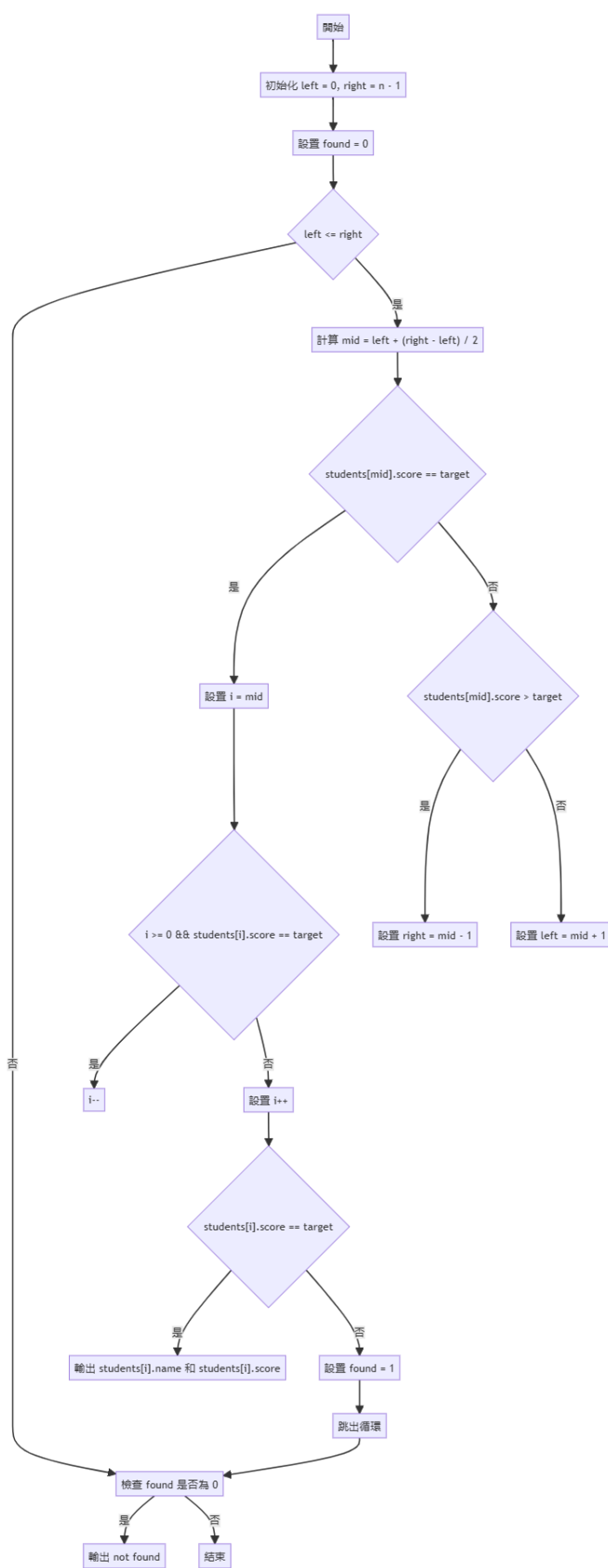


图 2-17 流程圖

3) 相关代码

相关代码 (19):

Listing 19 示例

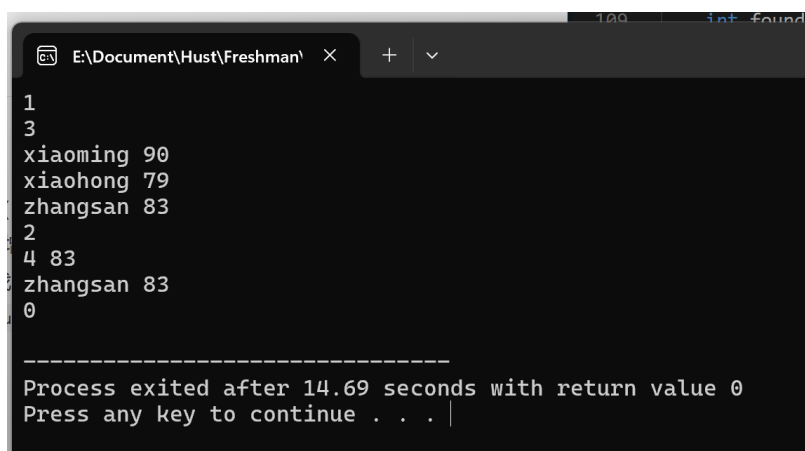
```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4      #define MAX_STUDENTS 100
5      #define MAX_NAME_LENGTH 50
6      typedef struct {
7          char name[MAX_NAME_LENGTH];
8          int score;
9      } Student;
10     void inputStudents(Student students[], int n);
11     void sortStudents(Student students[], int n);
12     void outputStudents(Student students[], int n);
13     void binarySearch(Student students[], int n, int target);
14     int main() {
15         Student students[MAX_STUDENTS];
16         int choice, n;
17         do {
18             scanf("%d", &choice);
19             ;
20             switch (choice) {
21                 case 1:
22                     scanf("%d", &n);
23                     inputStudents(students, n);
24                     break;
25                 case 2:
26                     sortStudents(students, n);
27                     break;
28                 case 3:
29                     outputStudents(students, n);
30                     break;
31                 case 4:
32                     {
33                         int target;
34                         scanf("%d", &target);
35                         binarySearch(students, n, target);
36                         break;
```

```
37         }
38         case 0:
39             break;
40     }
41     } while (choice!= 0);
42     return 0;
43 }
44 void inputStudents(Student students[], int n) {
45     for (int i = 0; i < n; i++) {
46
47         scanf("%s %d", students[i].name, &students[i].score);
48     }
49 }
50 void sortStudents(Student students[], int n) {
51     for (int i = 0; i < n - 1; i++) {
52         for (int j = i + 1; j < n; j++) {
53             if (students[i].score < students[j].score) {
54                 int tempScore = students[i].score;
55                 students[i].score = students[j].score;
56                 students[j].score = tempScore;
57                 char tempName[MAX_NAME_LENGTH];
58                 strcpy(tempName, students[i].name);
59                 strcpy(students[i].name,
60                     students[j].name);
61                 strcpy(students[j].name, tempName);
62             } else if (students[i].score ==
63                 students[j].score) {
64                 if (strcmp(students[i].name,
65                     students[j].name) > 0) {
66                     char tempName[MAX_NAME_LENGTH];
67                     strcpy(tempName,
68                         students[i].name);
69                     strcpy(students[i].name,
70                         students[j].name);
71                     strcpy(students[j].name,
72                         tempName);
73                     int tempScore =
74                         students[i].score;
75                     students[i].score =
76                         students[j].score;
77                     students[j].score = tempScore;
```

```
70         }
71     }
72 }
73 }
74 }
75 void outputStudents(Student students[], int n) {
76     for (int i = 0; i < n; i++) {
77         printf("%s %d\n", students[i].name,
78             &students[i].score);
79     }
80 }
81 void binarySearch(Student students[], int n, int target) {
82     int left = 0;
83     int right = n - 1;
84     int found = 0;
85     while (left <= right) {
86         int mid = left + (right - left) / 2;
87         if (students[mid].score == target) {
88             int i = mid;
89             while (i >= 0 && students[i].score == target)
90                 i--;
91             i++;
92             for (; i < n && students[i].score == target;
93                 i++) {
94                 printf("%s %d\n", students[i].name,
95                     students[i].score);
96             }
97             found = 1;
98             break;
99         } else if (students[mid].score > target) {
100             right = mid - 1;
101         } else {
102             left = mid + 1;
103         }
104     }
105     if (!found) {
106         printf("not found\n");
107     }
108 }
```

4) 执行结果

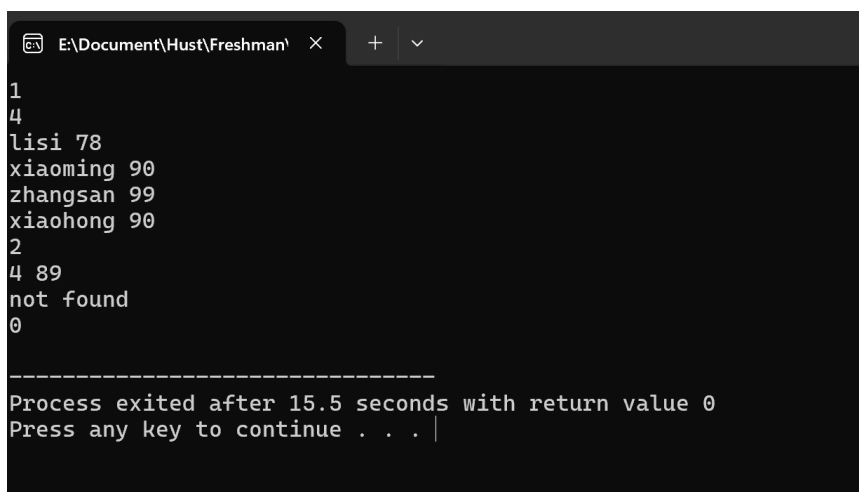
执行结果 (2-182-19):



```
E:\Document\Hust\Freshman\ > 1
E:\Document\Hust\Freshman\ > 3
E:\Document\Hust\Freshman\ > xiaoming 90
E:\Document\Hust\Freshman\ > xiaohong 79
E:\Document\Hust\Freshman\ > zhangsan 83
E:\Document\Hust\Freshman\ > 2
E:\Document\Hust\Freshman\ > 4 83
E:\Document\Hust\Freshman\ > zhangsan 83
E:\Document\Hust\Freshman\ > 0

-----
Process exited after 14.69 seconds with return value 0
Press any key to continue . . .
```

图 2-18 执行示例



```
E:\Document\Hust\Freshman\ > 1
E:\Document\Hust\Freshman\ > 4
E:\Document\Hust\Freshman\ > lisi 78
E:\Document\Hust\Freshman\ > xiaoming 90
E:\Document\Hust\Freshman\ > zhangsan 99
E:\Document\Hust\Freshman\ > xiaohong 90
E:\Document\Hust\Freshman\ > 2
E:\Document\Hust\Freshman\ > 4 89
E:\Document\Hust\Freshman\ > not found
E:\Document\Hust\Freshman\ > 0

-----
Process exited after 15.5 seconds with return value 0
Press any key to continue . . .
```

图 2-19 执行示例

这里可见是先操作了排序再查找，不然的话直接使用二分查找是不一定能找到结果的，这两幅图分别显示了查找成功和查找失败的输出结果，可见该程序是满足题目的要求的。

2.3.4 5-9 插入字符串

1) 目标内容

编写函数 `strnins(t,s,n)`, 其功能是：将串 `s` 插入串 `t` 串的第 `n` 个字符的后面，编写 `main` 函数测试 `strnins` 函数的正确性。

2) 分析

题目要求实现一个名为 `strnins` 的函数，它需要接收三个参数，分别是两个字符串 `t` 和 `s`，以及一个表示位置的整数 `n`。该函数的主要功能是把字符串 `s` 插入到字符串 `t` 中第 `n` 个字符的后面，最终改变字符串 `t` 的内容以体现插入操作后的结果。

总流程图 (2-20):

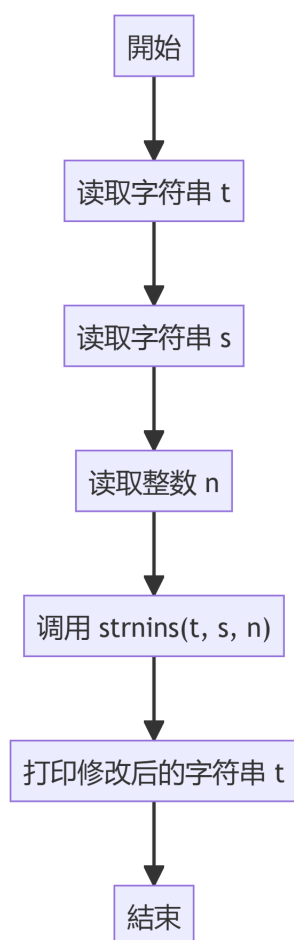


图 2-20 流程图

設計思路:

输入读取:

1. 从标准输入读取两个字符串 `t` 和 `s`。

2. 字符串插入逻辑:

计算字符串 `t` 和 `s` 的长度。

创建一个临时字符串 `temp`，其长度为 $\text{len_t} + \text{len_s} + 1$ ，以容纳合并后的字符串

和终止符 \0。

在 `temp` 的末尾添加终止符 \0。

将 `temp` 复制回 `t`。3. 输出结果

相关流程图 (2-21):

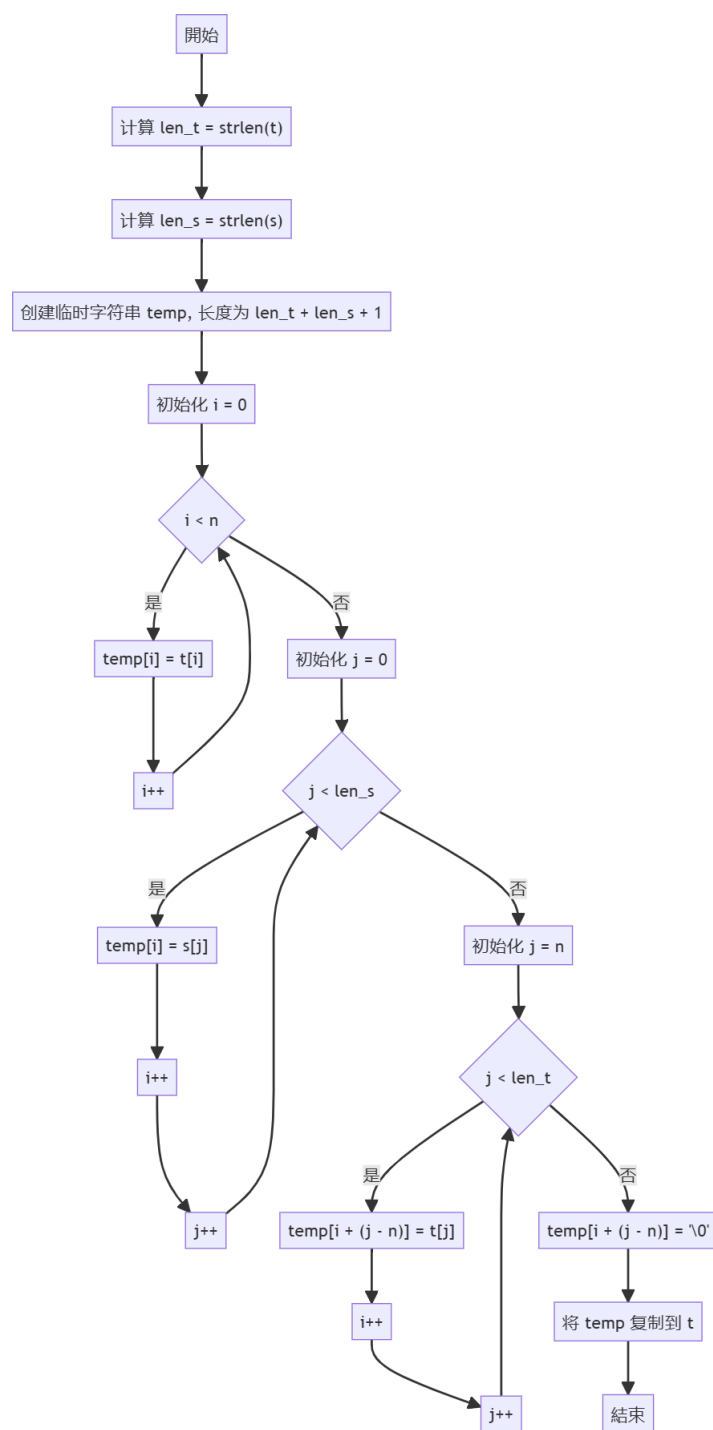


图 2-21 流程图

3) 相关代码

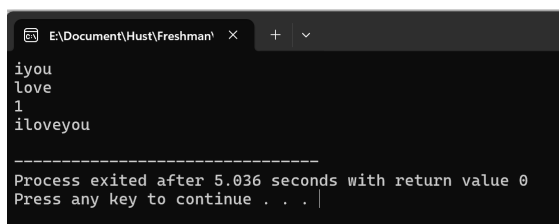
相关代码 (20):

Listing 20 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4      void strnins(char t[], char s[], int n);
5      int main() {
6          char t[100], s[100];
7          int n;
8          scanf("%s", t);
9          scanf("%s", s);
10         scanf("%d", &n);
11         strnins(t, s, n);
12         printf("%s\n", t);
13         return 0;
14     }
15     void strnins(char t[], char s[], int n) {
16         int len_t = strlen(t);
17         int len_s = strlen(s);
18         int i, j;
19         char temp[len_t + len_s + 1];
20         for (i = 0; i < n; i++) {
21             temp[i] = t[i];
22         }
23         for (j = 0; j < len_s; j++) {
24             temp[i++] = s[j];
25         }
26         for (j = n; j < len_t; j++) {
27             temp[i + (j - n)] = t[j];
28         }
29         temp[i + (j - n)] = '\0';
30         strcpy(t, temp);
31     }
```

4) 执行结果

执行结果 (2-22):



```
E:\Document\Hust\Freshman\ x + v
iyou
love
1
iyouloveyou

-----
Process exited after 5.036 seconds with return value 0
Press any key to continue . . .
```

图 2-22 执行结果

可见在原有字符串“iyou”上的第 1 位加插了字符串“love”，使其输出为“Iloveyou”

2.3.5 5-10 消除类游戏

1) 目标内容

消除类游戏是一种益智游戏，其核心规则是将一定的彼此相邻的相同元素配对消除。现给定一个 n 行 m 列的棋盘，棋盘中的每一个方格上放着一个棋子，每个棋子都有颜色，编号用 1-9 表示。当一行或一列上有连续 3 个或 3 个以上同色棋子时，这些棋子都被同时消除，对应的方格用 0 表示，请输出经过消除后的棋盘

2) 分析

本题的流程图十分简洁，是一般最直接的输入->处理->输出，难在中间处理的部分。

總流程圖 (2-23):

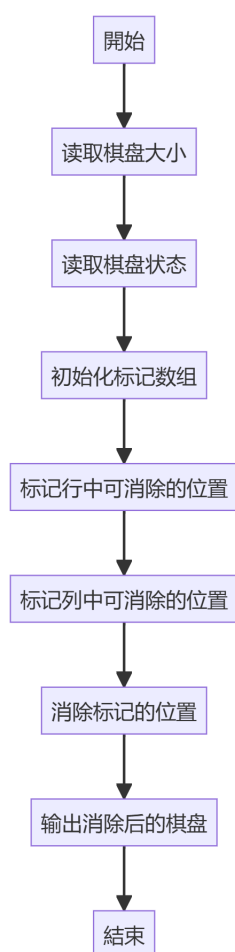


图 2-23 總流程圖

本题的核心在于识别棋盘上每行每列中连续出现 3 个或 3 个以上相同颜色（用 1 - 9 编号表示）的棋子，并将这些棋子消除，即将它们对应的方格标记为 0。这意味着需要逐行、逐列去检查棋子的颜色情况，判断是否满足消除条件。此题要构建这个棋盘，可利用二维数组 `int` 标识符 `[n][m]` 来表示棋盘

設計思路：

我们需要设计一个函数，该函数接收表示棋盘的二维数组 `board`、行数 `n` 和列数 `m` 作为参数，目的是对棋盘执行消除操作并返回消除后的棋盘状态。

在函数内部，通过循环嵌套实现对棋盘的逐行、逐列遍历，先检查每行是否有满足消除条件的棋子，再检查每列是否有满足消除条件的棋子，每次检查到满足条件的区域就执行消除操作（将对应方格标记为 0）。

由于消除部分棋子后可能会产生新的可消除区域，所以需要在一次完整的行、列遍历检查后，再次判断棋盘是否还有可消除的棋子，如果有则继续重复上述的检

查和消除过程，直到棋盘上不再有可消除的棋子为止。

这里处理矩阵时的代码比较复杂，主要可分为三部分：1. 标记行中可消除的位置
2. 标记列中可消除的位置 3. 消除标记的位置相關流程图 (2-242-25):

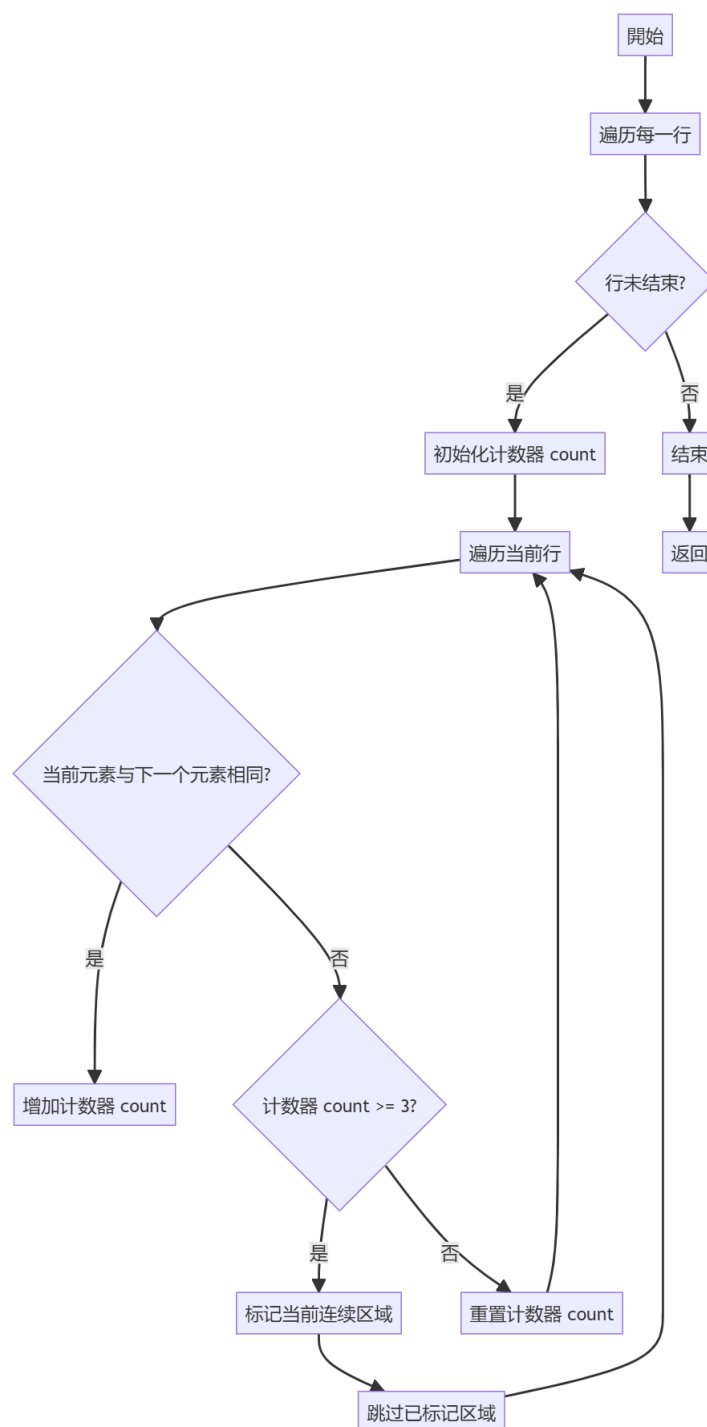


图 2-24 流程图

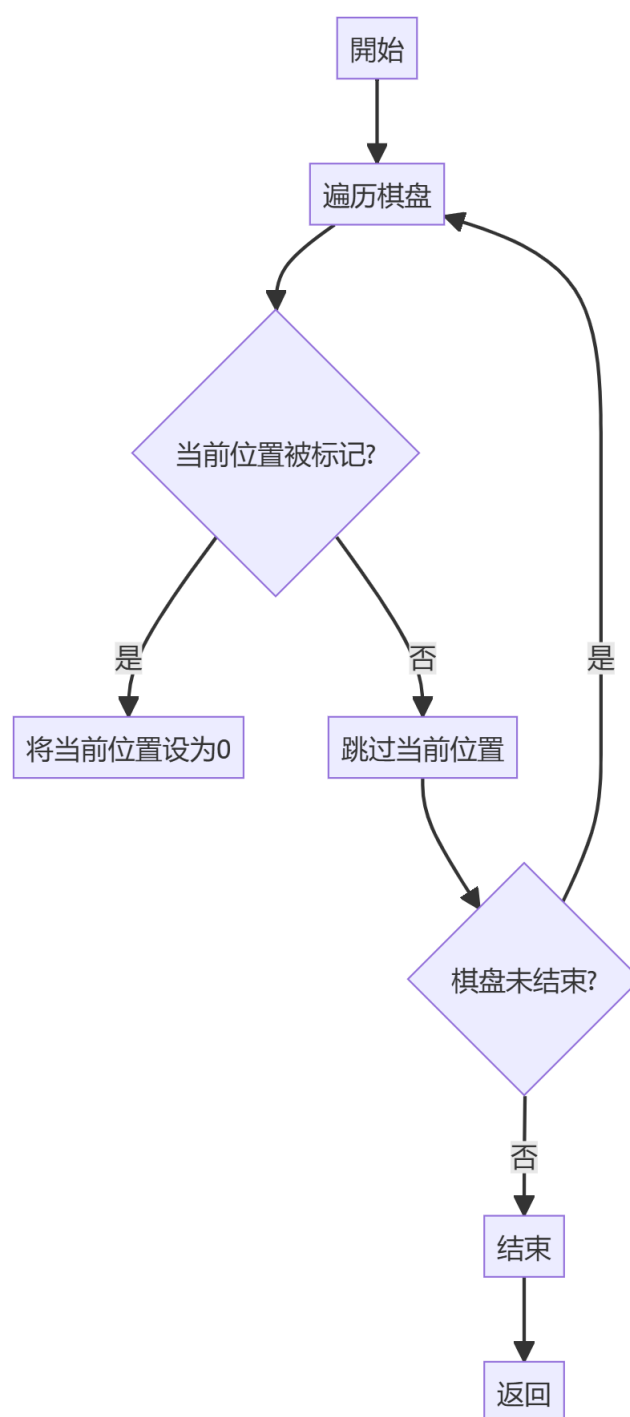


图 2-25 流程图

3) 相关代码

相关代码 (21):

Listing 21 示例

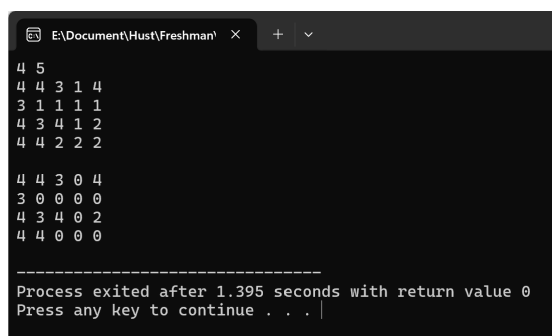
```
1      #include <stdio.h>
2      void markRowsToEliminate(int board[][100], int rows, int
           cols, int mark[][100]);
3      void markColumnsToEliminate(int board[][100], int rows, int
           cols, int mark[][100]);
4      void eliminateMarkedPositions(int board[][100], int rows,
           int cols, int mark[][100]);
5      int main() {
6          int board[100][100];
7          int rows, cols;
8          int i, j;
9          scanf("%d %d", &rows, &cols);
10         for (i = 0; i < rows; i++) {
11             for (j = 0; j < cols; j++) {
12                 scanf("%d", &board[i][j]);
13             }
14         }
15         int rowMark[100][100] = {0};
16         int colMark[100][100] = {0};
17         markRowsToEliminate(board, rows, cols, rowMark);
18         markColumnsToEliminate(board, rows, cols, colMark);
19         eliminateMarkedPositions(board, rows, cols, rowMark);
20         eliminateMarkedPositions(board, rows, cols, colMark);
21         for (i = 0; i < rows; i++) {
22             for (j = 0; j < cols; j++) {
23                 printf("%d", board[i][j]);
24                 if (j < cols - 1) {
25                     printf(" ");
26                 }
27             }
28             printf("\n");
29         }
30         return 0;
31     }
32     void markRowsToEliminate(int board[][100], int rows, int
           cols, int mark[][100]) {
33         int i, j, k;
34         for (i = 0; i < rows; i++) {
35             j = 0;
36             while (j < cols) {
37                 int count = 1;
```

```
38         k = j + 1;
39         while (k < cols && board[i][k] ==
40             board[i][j]) {
41             count++;
42             k++;
43         }
44         if (count >= 3) {
45             for (k = j; k < j + count; k++) {
46                 mark[i][k] = 1;
47             }
48             j = k;
49         }
50     }
51 }
52 void markColumnsToEliminate(int board[][100], int rows, int
53     cols, int mark[][100]) {
54     int i, j, k;
55     for (j = 0; j < cols; j++) {
56         i = 0;
57         while (i < rows) {
58             int count = 1;
59             k = i + 1;
60             while (k < rows && board[k][j] ==
61                 board[i][j]) {
62                 count++;
63                 k++;
64             }
65             if (count >= 3) {
66                 for (k = i; k < i + count; k++) {
67                     mark[k][j] = 1;
68                 }
69             }
70             i = k;
71         }
72     }
73 }
74 void eliminateMarkedPositions(int board[][100], int rows,
75     int cols, int mark[][100]) {
76     int i, j;
77     for (i = 0; i < rows; i++) {
```

```
75         for (j = 0; j < cols; j++) {
76             if (mark[i][j] == 1) {
77                 board[i][j] = 0;
78             }
79         }
80     }
81 }
```

4) 执行结果

执行结果 (2-262-27):

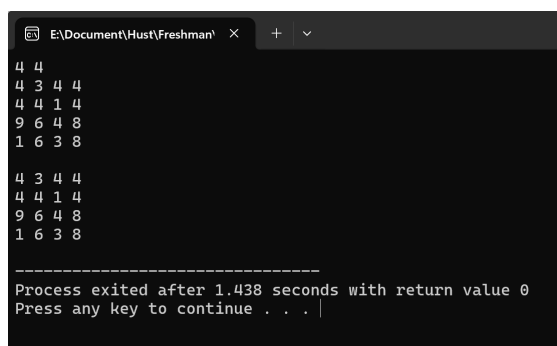


```
E:\Document\Hust\Freshman' x + v
4 5
4 4 3 1 4
3 1 1 1 1
4 3 4 1 2
4 4 2 2 2

4 4 3 0 4
3 0 0 0 0
4 3 4 0 2
4 4 0 0 0

-----
Process exited after 1.395 seconds with return value 0
Press any key to continue . . . |
```

图 2-26 执行示例



```
E:\Document\Hust\Freshman' x + v
4 4
4 3 4 4
4 4 1 4
9 6 4 8
1 6 3 8

4 3 4 4
4 4 1 4
9 6 4 8
1 6 3 8

-----
Process exited after 1.438 seconds with return value 0
Press any key to continue . . . |
```

图 2-27 执行示例

可以见到代码也是成功被执行的，能够实现矩阵的消除，通过遍历的过作去完成，先对位置标记再进行消除操作是十分关键的，之前自己刚开始写的时候没写好主要就是因为太急消除，这时候会阻碍了行 (列) 的消除，使得输出结果有误。因此，在编程时是十分注重程序的执行顺序和思想的。

2.3.6 5-11 N 阶顺转方阵

1) 目标内容

本关任务：将 1、2、3、 $\dots$ 、 $n*n$ 从左上角开始，由外层至中心按顺时针方向螺旋排列所形成的数字方阵，成为 n 阶顺转方阵，5 阶顺转方阵如下方数字阵列所示。编写程序，读入 n ，构造并输出 n 阶顺转方阵。

2) 分析

这段 C 语言代码实现生成并输出 n 阶顺转方阵的功能。在 `main` 函数里获取用户输入的 n 值，调用 `void` 函数按顺时针螺旋方式将 1 到 n^2 的数字填充进二维数组 `matrix`，利用边界变量控制填充顺序，每填充完一层对应边界变量调整，循环直至填完所有数字。之后通过 `void` 定义的函数将生成好的方阵以矩阵格式输出展示，整体完成按要求构造并输出 n 阶顺转方阵的任务。

总流程图 (2-28)：

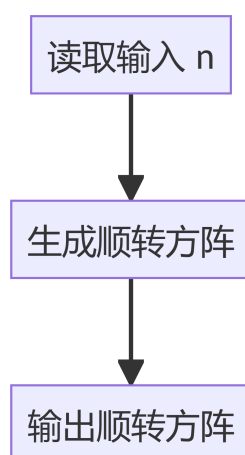


图 2-28 总流程图

设计思路：

本题整体采用模块化设计，在主函数中先定义变量接收用户输入的方阵阶数 n 及用于存储方阵的二维数组，接着调用生成方阵的函数，按顺时针螺旋方式，利用边界变量控制，从外层到内层逐次填充 1 到 n^2 的数字到数组，每填充完一层相应调整边界变量，直至填满。之后调用输出函数，通过两层嵌套循环遍历数组，输出各元素并按需添加空格和换行，将生成好的 n 阶顺转方阵展示出来，以此实现构造并输出 n 阶顺转方阵的功能。流程图 (2-29)：

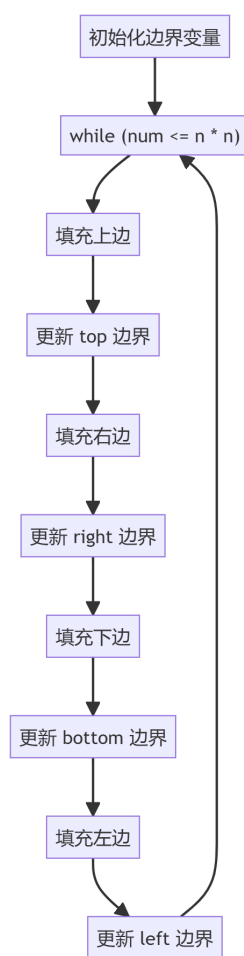


图 2-29 流程图

3) 相关代码

相关代码 (22):

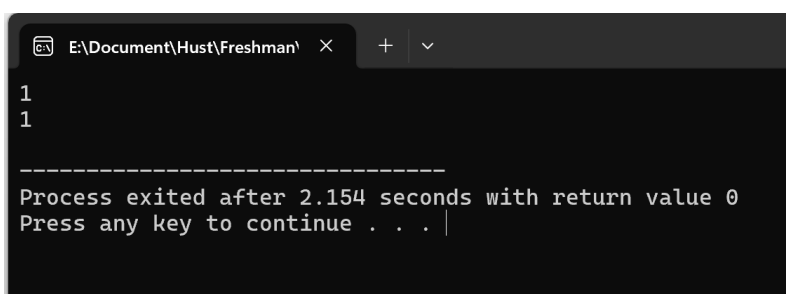
Listing 22 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      void generateSpiralMatrix(int n, int matrix[][100]);
4      void printMatrix(int n, int matrix[][100]);
5      int main() {
6          int n;
7          int matrix[100][100];
8          scanf("%d", &n);
9          generateSpiralMatrix(n, matrix);
10         printMatrix(n, matrix);
11         return 0;
```

```
12         }
13     void generateSpiralMatrix(int n, int matrix[][100]) {
14         int top = 0, bottom = n - 1, left = 0, right = n - 1;
15         int num = 1;
16         while (num <= n * n) {
17             for (int i = left; i <= right; i++) {
18                 matrix[top][i] = num++;
19             }
20             top++;
21             for (int i = top; i <= bottom; i++) {
22                 matrix[i][right] = num++;
23             }
24             right--;
25             for (int i = right; i >= left; i--) {
26                 matrix[bottom][i] = num++;
27             }
28             bottom--;
29             for (int i = bottom; i >= top; i--) {
30                 matrix[i][left] = num++;
31             }
32             left++;
33         }
34     }
35     void printMatrix(int n, int matrix[][100]) {
36         for (int i = 0; i < n; i++) {
37             for (int j = 0; j < n; j++) {
38                 printf("%d", matrix[i][j]);
39                 if (j < n - 1) {
40                     printf(" ");
41                 }
42             }
43             printf("\n");
44         }
45     }
```

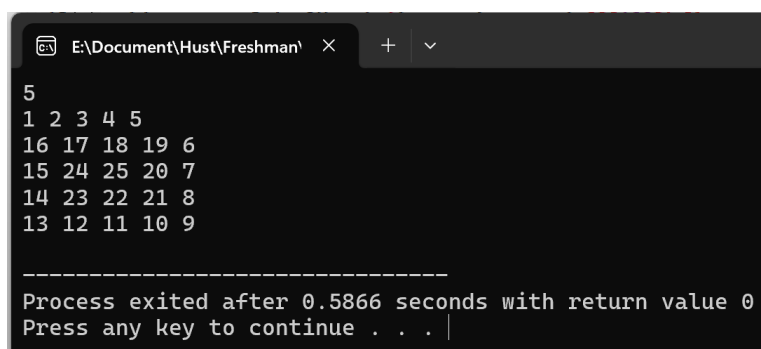
4) 执行结果

执行结果 (2-302-31):



```
E:\Document\Hust\Freshman> 1
1
-----
Process exited after 2.154 seconds with return value 0
Press any key to continue . . . |
```

图 2-30 执行示例



```
E:\Document\Hust\Freshman> 5
1 2 3 4 5
16 17 18 19 6
15 24 25 20 7
14 23 22 21 8
13 12 11 10 9
-----
Process exited after 0.5866 seconds with return value 0
Press any key to continue . . . |
```

图 2-31 执行示例

由此可见该程序能够成功输出 N 阶顺转矩阵。

填充上侧一行：

使用一个 for 循环，从左边界 left 到右边界 right 遍历，将当前的数字 num 依次填充到 matrix[top][i] 中（其中 i 为循环变量），每填充一个位置，num 的值就自增 1，代表下一个要填充的数字。完成这一行的填充后，将上边界 top 的值加 1，准备填充下一层的右侧一列。

填充右侧一列：

接着再用一个 for 循环，从上边界 top 到下边界 bottom 遍历，把 num 依次填充到 matrix[i][right] 中，同样每填充一次 num 自增 1，填充完这一列后，将右边界 right 的值减 1，以便后续填充下一层的下侧一行。

填充下侧一行：

然后通过一个 for 循环，从右边界 right 到左边界 left 反向遍历，将 num 填充到 matrix[bottom][i] 中（注意这里 i 是反向变化的），num 持续自增，填充完这一行后，下边界 bottom 的值减 1，为填充下一层的左侧一列做准备。

填充左侧一列：

最后利用一个 for 循环，从下边界 bottom 到上边界 top 反向遍历，把 num 填充到

`matrix[i][left]` 中, `num` 继续自增, 填充完这一列后, 左边界 `left` 的值加 1, 至此完成一层的填充, 进入下一轮 `while` 循环, 重复上述步骤填充下一层, 直到整个方阵所有位置都填充完毕。通过两层嵌套的 `for` 循环来遍历二维数组 `matrix`, 外层循环控制行索引, 内层循环控制列索引。对于每个元素 `matrix[i][j]`, 先使用 `printf` 函数输出其数值, 然后通过判断 `j < n - 1` 来确定是否为该行的最后一个元素, 如果不是, 则输出一个空格进行分隔, 使得每行元素之间有合适的间隔。当内层循环结束, 也就是一行元素都输出完后, 使用 `printf` 函数输出换行符, 开始下一行元素的输出, 如此循环, 直至整个方阵的所有元素都被输出, 让用户能直观地看到生成的顺转方阵的样子。

2.3.7 5-12 迷宫问题

1) 目标内容

迷宫问题。编程找出从入口（左上角方格）经过迷宫到达出口（右下角方格）的所有路径, 迷宫问题示意如下方数字阵列所示, 0 所表示的地方是不可以通行的, 只能从 1 走到与它相邻（四邻域: 上、下、左、右相邻）的 1 上, 且不能走重复路径。

2) 分析

要解决迷宫问题, 即找出从迷宫入口（左上角方格）到出口（右下角方格）的所有可行路径。迷宫用二维数组表示, 其中 0 代表不可通行区域, 1 代表可通行区域。代码通过深度优先搜索（DFS）算法来遍历迷宫, 记录下每条从入口到出口的路径, 并输出这些路径

总流程图 (2-32):

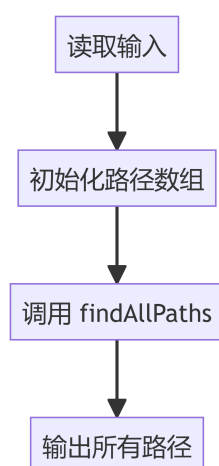


图 2-32 总流程图

设计思路:

首先我们要创建一个同样大小的二维数组 `path`，初始化为 0。定义起点 `start` 和终点 `end`。调用 `dfs` 函数从起点开始进行深度优先搜索。

这里说明一下核心 DFS 算法的设计思路

DFS（深度优先搜索）函数设计思路：

首先进行边界和可行性判断，如果当前位置超出迷宫范围（通过坐标与行数、列数比较判断）、当前位置是不可通行的（`maze[current.x][current.y] == 0`）或者已经走过（`path[current.x][current.y] == 1`），则直接返回，不再继续探索该位置。

若当前位置合法且可通行，将当前位置在 `path` 数组中标记为 1，表示已走过。接着判断当前位置是否就是终点，如果是，则说明找到了一条从入口到出口的路径，此时路径数量 `pathCount` 加 1，输出路径编号以及通过调用 `printPath` 函数输出该条路径的具体情况（用 1 和 0 展示走过和未走过的方格），然后将当前位置标记回 0（回溯操作，方便后续探索其他路径时该位置能被再次访问），并返回。

如果当前位置不是终点，就分别向当前位置的上、下、左、右四个方向（通过创建对应方向的 `Point` 结构体来表示新位置）进行递归调用 `dfs` 函数，继续深度优先搜索，探索从这些相邻位置能否到达终点，每探索完一个方向后，同样进行回溯操作（将当前位置标记为 0），确保能遍历所有可能的路径情况。

相关流程图 (2-33):

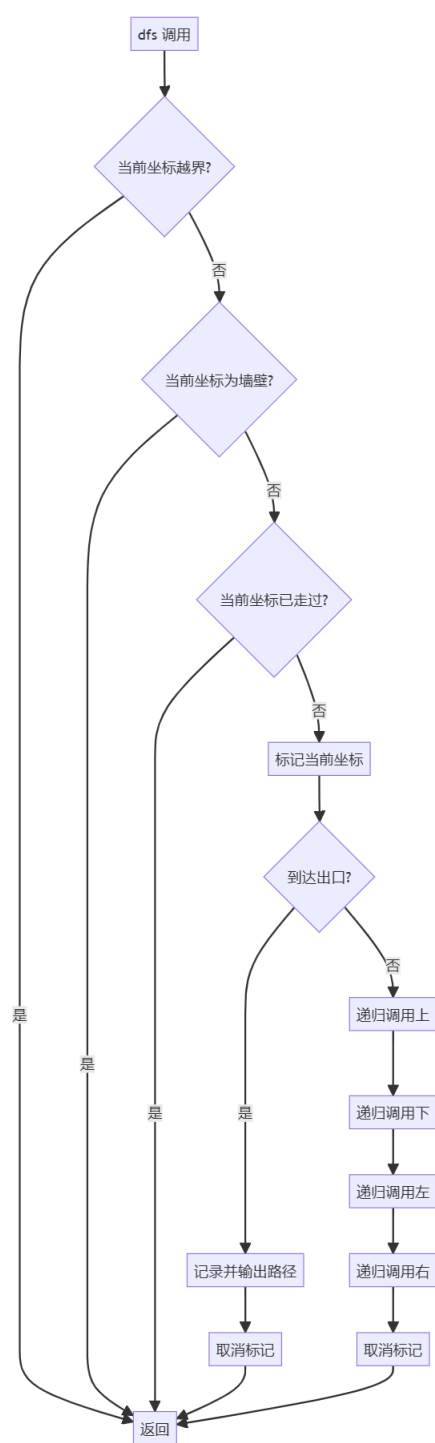


图 2-33 流程图

3) 相关代码

相关代码 (23):

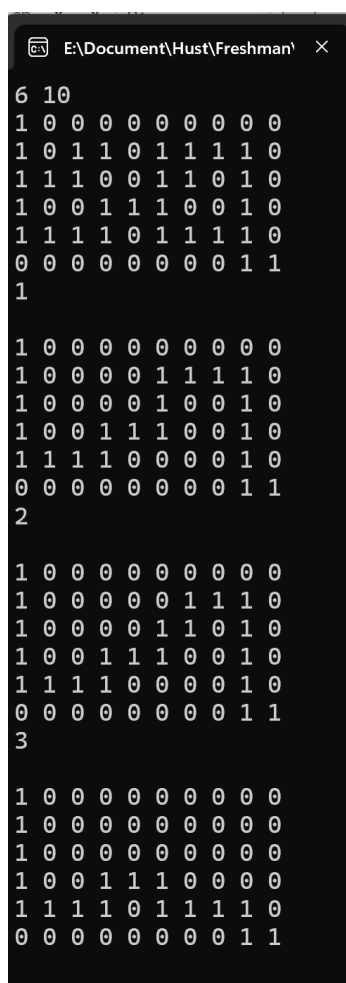
Listing 23 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #define MAX_ROWS 100
4      #define MAX_COLS 100
5      typedef struct {
6          int x;
7          int y;
8      } Point;
9      int pathCount = 0;
10     void findAllPaths(int maze[MAX_ROWS][MAX_COLS], int rows,
11                      int cols);
12     void printPath(int path[MAX_ROWS][MAX_COLS], int rows, int
13                   cols);
14     void dfs(int maze[MAX_ROWS][MAX_COLS], int
15             path[MAX_ROWS][MAX_COLS], Point current, Point end, int
16             rows, int cols);
17     int main() {
18         int maze[MAX_ROWS][MAX_COLS];
19         int rows, cols;
20         int i, j;
21         scanf("%d %d", &rows, &cols);
22         for (i = 0; i < rows; i++) {
23             for (j = 0; j < cols; j++) {
24                 scanf("%d", &maze[i][j]);
25             }
26         }
27         findAllPaths(maze, rows, cols);
28         return 0;
29     }
30     void findAllPaths(int maze[MAX_ROWS][MAX_COLS], int rows,
31                      int cols) {
32         int path[MAX_ROWS][MAX_COLS];
33         Point start = {0, 0};
34         Point end = {rows - 1, cols - 1};
35         for (int i = 0; i < rows; i++) {
36             for (int j = 0; j < cols; j++) {
37                 path[i][j] = 0;
38             }
39         }
40         dfs(maze, path, start, end, rows, cols);
41     }
```

```
37 void dfs(int maze[MAX_ROWS][MAX_COLS], int
    path[MAX_ROWS][MAX_COLS], Point current, Point end, int
    rows, int cols) {
38     if (current.x < 0 || current.x >= rows || current.y
        < 0 || current.y >= cols ||
39     maze[current.x][current.y] == 0 ||
        path[current.x][current.y] == 1) {
40         return;
41     }
42     path[current.x][current.y] = 1;
43     if (current.x == end.x && current.y == end.y) {
44         pathCount++;
45         printf("%d\n", pathCount);
46         printPath(path, rows, cols);
47         path[current.x][current.y] = 0;
48         return;
49     }
50     Point up = {current.x - 1, current.y};
51     dfs(maze, path, up, end, rows, cols);
52     Point down = {current.x + 1, current.y};
53     dfs(maze, path, down, end, rows, cols);
54     Point left = {current.x, current.y - 1};
55     dfs(maze, path, left, end, rows, cols);
56     Point right = {current.x, current.y + 1};
57     dfs(maze, path, right, end, rows, cols);
58     path[current.x][current.y] = 0;
59 }
60 void printPath(int path[MAX_ROWS][MAX_COLS], int rows, int
    cols) {
61     for (int i = 0; i < rows; i++) {
62         for (int j = 0; j < cols; j++) {
63             printf("%d", path[i][j]);
64             if (j < cols - 1) {
65                 printf(" ");
66             }
67         }
68         printf("\n");
69     }
70 }
```

4) 执行结果

执行结果 (2-34):



```
E:\Document\Hust\Freshman
6 10
1 0 0 0 0 0 0 0 0 0
1 0 1 1 0 1 1 1 1 0
1 1 1 0 0 1 1 0 1 0
1 0 0 1 1 1 0 0 1 0
1 1 1 1 0 1 1 1 1 0
0 0 0 0 0 0 0 0 1 1
1
1 0 0 0 0 0 0 0 0 0
1 0 0 0 0 1 1 1 1 0
1 0 0 0 0 1 0 0 1 0
1 0 0 1 1 1 0 0 1 0
1 1 1 1 0 0 0 0 1 0
0 0 0 0 0 0 0 0 1 1
2
1 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 1 1 1 0
1 0 0 0 0 1 1 0 1 0
1 0 0 1 1 1 0 0 1 0
1 1 1 1 0 0 0 0 1 0
0 0 0 0 0 0 0 0 1 1
3
1 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0
1 0 0 0 0 0 0 0 0 0
1 0 0 1 1 1 0 0 0 0
1 1 1 1 0 1 1 1 1 0
0 0 0 0 0 0 0 0 1 1
```

图 2-34 执行示例

首先需要把迷宫抽象成二维数组的形式，用 0 和 1 来清晰地表示不可通行区域和可通行区域，这一步骤是后续进行算法设计和代码编写的基础。将现实问题转化为计算机能够处理的数据结构，让整个问题变得更有条理，便于操作和思考解决办法。

明确了起点和终点的坐标表示，进一步把寻找路径的问题聚焦到如何从给定的起点出发，通过合理的探索规则去尝试抵达终点，这种明确的问题定义和模型建立有助于确定具体的解题方向。

DFS 是一种用于遍历或搜索图（在迷宫问题里可以把迷宫看作是一种特殊的图结构，方格就是节点，相邻方格之间的可通行关系就是边）、树等数据结构的算法。它的核心思想是沿着一条路径尽可能深地探索下去，直到无法继续前进（比

如遇到边界、不符合条件的节点或者已经访问过的节点等情况), 然后回溯到上一个未完全探索的节点, 再继续沿着另一条未探索的分支路径深入, 如此反复, 直到遍历完所有可能的路径。

这种思路能够暴力破解问题, 但同时所需要的运行时间亦是巨大的, 因此掌握熟练多种算法的思路往往是必不可少的。

2.4 实验五小结

在完成诸多关于数组的实验题过程中, 我不仅对数组这一基础数据结构有了更深入透彻的理解与掌握, 还接触并运用到了大量数组以外的知识和算法, 极大地拓宽了我的编程视野与思维方式。以下是我对此次数组实验题学习经历的详细总结与深刻反思。

2.4.1 知识与技能提升

(一) 数组基础的巩固与深化

数组作为一种存储相同类型数据的线性数据结构, 在实验题中频繁出现, 使我对其特性和操作有了更熟练的运用。通过对数组元素的访问、修改、遍历等基本操作的大量实践, 我深刻理解了数组下标与内存地址之间的对应关系, 明白了如何通过合理运用下标来高效地操作数组中的数据。例如, 在处理一些涉及数据统计和筛选的问题时, 能够灵活地运用循环结构和数组下标, 快速定位并处理目标元素, 提高了程序的执行效率。

(二) 与其他数据结构的关联与拓展

在部分实验题中, 数组与其他数据结构(如结构体)相互结合使用, 展现出了更强大的数据组织和处理能力。例如, 通过定义结构体数组来存储具有多个属性的数据对象, 不仅可以方便地对每个对象的各个属性进行单独操作, 还能够以数组的形式对一组相关对象进行整体处理。这种结合方式使我认识到不同数据结构之间并非孤立存在, 而是可以相互协作, 共同解决复杂的编程问题。在处理涉及多个相关数据实体且需要批量处理的场景时, 这种数据结构的组合应用尤为重要, 它让我学会从更宏观的角度去思考数据的组织和管理, 提升了我构建复杂数据模型的能力。

(三) 算法在数组中的应用

实验题中广泛涉及各种算法在数组上的应用, 如排序算法(冒泡排序、选择排序、

快速排序等)、查找算法(线性查找、二分查找等)以及遍历算法(深度优先遍历、广度优先遍历的变体等)。这些算法的应用让我明白了如何根据具体问题的需求选择合适的算法策略来对数组数据进行处理,以达到优化程序性能的目的。

排序算法:在处理数组排序问题时,我深入学习了不同排序算法的原理和实现方式。冒泡排序虽然简单易懂,但时间复杂度较高,适用于小规模数据的排序;而快速排序则采用了分治思想,通过选择一个基准元素将数组分为两部分,递归地对两部分进行排序,平均时间复杂度较低,在处理大规模数据时表现出色。通过对这些排序算法的实践,我不仅能够根据数据规模 and 实际需求选择最合适的排序方法,还学会了对排序算法进行简单的优化,如在冒泡排序中加入标记位来提前结束排序过程,或者在快速排序中合理选择基准元素以避免最坏情况的发生。这些优化技巧不仅提高了程序的运行效率,也培养了我编程过程中不断追求性能提升的意识。

查找算法:对于数组查找问题,线性查找适用于无序数组,但时间复杂度为 $O(n)$;二分查找则要求数组必须有序,但其时间复杂度可降低至 $O(\log n)$ 。在实际应用中,我学会了根据数组的有序性和查找需求来选择合适的查找算法。例如,在处理频繁查找操作且数组相对稳定不变的情况下,先对数组进行排序,然后使用二分查找可以大大提高查找效率。此外,我还了解到一些基于哈希表的查找算法,它们在特定场景下能够实现更快速的查找操作,但需要额外的空间来存储哈希表。通过对这些查找算法的学习和实践,我深刻认识到在编程中权衡时间和空间复杂度的重要性,以及根据具体问题场景选择最优算法策略的必要性。

遍历算法:在涉及数组元素的遍历和搜索场景中,我运用了深度优先遍历和广度优先遍历的思想。例如,在处理二维数组表示的迷宫问题时,通过深度优先遍历算法来探索从起点到终点的所有可能路径,利用递归函数来实现深度优先遍历的过程,同时通过标记已访问的数组元素来避免重复访问,提高算法效率。而在一些需要按照层次顺序处理数组元素的问题中,如树状结构的层次遍历模拟题,我则运用了广度优先遍历的思想,借助队列数据结构来实现元素的层次顺序访问。这些遍历算法的应用让我更加熟练地掌握了递归和队列等编程技术,同时也培养了我面对不同问题场景时灵活选择合适算法策略的能力。

2.4.2 问题解决的思路多样化

(一) 复杂问题的分析与拆解

面对实验题中那些较为复杂的问题,我逐渐学会了运用分析和拆解的方法将其

转化为一系列可管理和解决的子问题。例如，在处理涉及多个条件限制和多种数据处理步骤的数组问题时，我首先会仔细梳理问题的需求和给定的条件，确定问题的关键目标和核心难点。然后，将整个问题按照功能模块或数据处理流程拆分成若干个相对独立的子问题，每个子问题专注于完成特定的任务或处理特定部分的数据。这种分析和拆解的思维方式使我在面对复杂编程问题时不再感到无从下手，而是能够有条不紊地逐步推进解决方案的设计和实现。通过将大问题分解为小问题，我不仅降低了问题的解决难度，还提高了代码的可读性和可维护性，因为每个子问题的解决代码相对独立且功能明确，便于后续的调试和修改。

（二）逻辑思维与编程思维的强化

在解决数组实验题的过程中，我的逻辑思维和编程思维得到了极大的锻炼和强化。每一道实验题都像是一个逻辑谜题，需要我运用严谨的逻辑推理来设计合理的算法和数据处理流程。在编写代码之前，我会先在脑海中构建出问题的解决方案框架，考虑各种可能的情况和边界条件，并设计相应的应对策略。这种逻辑思考和规划的过程让我学会了从抽象的问题描述逐步转化为具体的编程实现步骤，培养了我结构化编程的思维方式。同时，在代码编写过程中，我需要不断地调试和优化代码，以确保程序的正确性和高效性。这一过程锻炼了我发现问题、定位问题和解决问题的能力，使我能够更加敏锐地察觉到代码中的逻辑错误和潜在的性能瓶颈，并通过合理的代码调整和算法优化来加以解决。通过反复的实践和积累，我的逻辑思维和编程思维逐渐变得更加严谨、灵活和高效，能够更好地应对各种复杂编程任务的挑战。

（三）调试技巧与错误处理能力的提升

调试是编程过程中不可或缺的重要环节，在数组实验题的实践中，我积累了丰富的调试技巧和错误处理经验。当程序出现错误或异常结果时，我学会了首先仔细观察程序的输出和运行时的表现，通过分析错误信息和程序的执行流程来初步定位问题可能出现的区域。然后，我会运用调试工具（如断点调试、变量监视等）来深入检查程序在关键位置的变量值和执行状态，逐步缩小问题的范围，直至找到错误的根源。在处理数组相关的错误时，常见的问题包括数组下标越界、数组初始化错误、算法逻辑错误导致的数组元素处理不当等。针对这些问题，我学会了在编写代码时加入必要的边界检查和错误处理机制，以提高程序的健壮性。例如，在访问数组元素之前，先检查下标是否在合法范围内；在进行数组排序或查找操作时，确保数组不为空且满足相应的前提条件。通过不断地调试和优

化程序，我不仅能够更加熟练地运用调试工具和技巧，还培养了良好的错误预防和处理意识，使我编写的程序能够更加稳定可靠地运行。

2.4.3 心得与感悟

通过大量数组实验题的练习，我深刻体会到编程是一门需要不断实践才能真正掌握的艺术。理论知识固然重要，但只有通过实际动手编写代码，将所学的知识应用到具体的问题解决中，才能真正理解和内化这些知识。在实践过程中，我遇到了各种各样的问题和挑战，每一次解决问题的过程都是一次学习和成长的机会。从最初的对一些简单数组操作的生疏和错误频出，到后来能够熟练运用数组及相关知识解决复杂的编程问题，这种能力的提升离不开大量的实践积累。编程不仅仅是按照语法规则编写代码，更是一种创造性的活动，需要在实践中不断尝试新的算法策略、优化代码结构、提高程序性能，才能编写出高质量、高效率的程序代码。

只有不断学习，才能进步，只有不断实践，才能运用自如。

3 实验七结构与联合

3.1 程序改错与跟踪调试

3.1.1 7-1 表达式求值的程序验证

1) 目标内容

设有下面说明, 请先自己计算表 7-1 中表达式的值, 然后通过编程计算来加以验证。各表达式之间相互无关
需要验证的相关程序 (24):

Listing 24 示例

```
1      #include<stdio.h>
2      #include<stdlib.h>
3      char u[]="UVWXYZ",v[]="xyz";
4      struct T{
5          int x;
6          char c;
7          char *t;
8      }a[]={11, 'A', u}, {100, 'B', v}, *p=a;
9      int main()
10     {
11         int n;
12         scanf("%d",&n);
13         switch (n){
14             case 1:
15                 printf("%d", (++p)->x);
16                 break;
17             case 2:
18                 p++;
19                 printf("%c", p->c);
20                 break;
21             case 3:
22                 p++;
23                 printf("%c", *p->t);
24                 break;
25             case 4:
26                 printf("%c", *(++p)->t);
27                 break;
```

```
28         case 5:
29             printf("%c", **p->t);
30             break;
31         case 6:
32             printf("%c", **p->t);
33             break;
34     }
35     return 0;
36 }
```

2) 分析

对不同分模功能说明：

Case1

在这个 case 中，执行了 $(++p) \rightarrow x$ 操作。 $++p$ 会使指针 p 自增，也就是让 p 指向结构体数组 a 的下一个元素，即 $a[1]$ 。然后通过 $\rightarrow$ 运算符访问 p 所指向结构体（此时是 $a[1]$ ）的成员 x ，并将其值输出。由于 $a[1].x$ 的值初始化为 100，所以此 case 执行后会输出 100。

Case2

首先执行 $p++$ ，同样使指针 p 指向下一个结构体元素（也就是 $a[1]$ ），这和 case 1 中指针 p 的最终指向是一样的。然后通过 $p \rightarrow c$ 访问 p 所指向结构体（ $a[1]$ ）的成员 c ，并输出该字符值。因为 $a[1].c$ 的值为 'B'，所以此 case 执行后会输出 'B'。

Case3

先是 $p++$ ，让 p 指向 $a[1]$ 。然后 $*p \rightarrow t$ 这个表达式的运算顺序是先通过 $p \rightarrow t$ 获取 p 所指向结构体（即 $a[1]$ ）中成员 t 的值，此时 $p \rightarrow t$ 指向字符数组 v （因为 $a[1].t$ 指向 v ），再对这个指针进行解引用 $*$ ，就会获取到 v 数组的第一个字符，也就是 'x'，所以此 case 执行后会输出 'x'。

Case4

这里先执行 $(++p) \rightarrow t$ ， $++p$ 使 p 指向 $a[1]$ ，然后获取 $a[1].t$ ，也就是指向字符数组 v 的指针。接着再对这个指针进行解引用 $*$ ，同样获取到 v 数组的第一个字符 'x'，所以此 case 执行后输出 'x'，和 case 3 的输出结果一样，不过这里是先对指针 p 进行自增操作，再获取并解引用对应的指针成员，运算顺序有所不同。

Case5

对于表达式 $*++p \rightarrow t$ ，根据运算符优先级，先计算 $p \rightarrow t$ ，此时 p 如果还指向初始位置（也就是 $a[0]$ ），那么 $p \rightarrow t$ 指向字符数组 u ，接着执行 $++$ 操作，会使指针

`p->t` 自增，也就是让它指向 `u` 数组的下一个字符（即‘V’），最后再通过 `*` 解引用这个自增后的指针，所以此 `case` 执行后会输出‘V’。需要注意的是，这里容易和 `case 4` 的运算顺序混淆，关键区别在于 `++` 作用的对象不同，这里是作用在指针 `p->t` 上

Case6

表达式 `++*p->t` 的运算顺序是，先通过 `p->t` 获取指针所指向的字符（假设 `p` 初始指向 `a[0]`，那么 `p->t` 指向 `u` 数组的第一个字符‘U’），然后对这个获取到的字符进行自增操作 `++`，也就是将‘U’的 ASCII 码值加 1，变为‘V’，最后输出这个自增后的字符，所以此 `case` 执行后会输出‘V’。这里是对指针所指向的字符进行自增操作

3.2 程序完善与修改替换

3.2.1 7-2 源程序修改替换（一）

1) 目标内容

给定一批整数, 以 0 作为结束标志且不作为结点, 将其建成一个先进先出的链表, 先进先出链表的头指针始终指向最先创建的结点 (链头), 先建结点指向后建结点, 后建结点始终是尾结点。(1) 源程序中存在什么样的错误 (先观察执行结果)? 对程序进行修改、调试, 使之能够正确完成指定任务。

原程序 (25):

Listing 25 示例

```
1      #include<stdio.h>
2      #include<stdlib.h>
3      struct s_list
4      {
5          int data;
6          struct s_list *next;
7      };
8      void create_list (struct s_list *headp, int *p);
9      int main (void){
10         struct s_list *head =NULL,*p;
11         int s[] ={1,2,3,4,5,6,7,8,0};
12         create_list(head,s); //错误点1
13         p=head;
```

```
14         while (p){
15             printf("%d\t",p->data);
16             p=p->next;
17         }
18         printf("\n");
19         return 0;
20     }
21     void create_list(struct s_list * headp, int *p) //错误点2
22     {
23         struct s_list * loc_head=NULL,*tail; //错误点3
24         if (p[0]==0)
25             ;
26         else{
27             loc_head =(struct s_list* )malloc(sizeof
28                 (struct s_list));
29             loc_head->data=*p++;
30             tail=loc_head;
31             while (* p){ //错误点4
32                 tail->next =(struct s_list *)malloc
33                     (sizeof (struct s_list));
34                 tail =tail->next;
35                 tail->data = *p++;
36             }
37             tail->next =NULL;
38         }
39         headp =loc_head; //错误点5
```

2) 分析

错误点 1:

此处为值传递，在 create_list 函数内对 headp 的修改无法影响到 main 函数中的 head 变量

错误点 2:

应该使用二级指针来接收头指针的地址，以便能修改实参的值

错误点 3:

tail 变量未初始化就使用，虽然部分编译器可能不报错，但不符合规范，可能导致潜在错误

错误点 4:

循环结束条件判断错误，本意是判断当前指向的整数是否为 0 来决定是否结束

循环创建链表节点，但这样只要 p 指向的内存内容非零就会继续循环，不符合预期

错误点 5:

由于前面参数传递是值传递，这里只是修改了形参 headp 的值，无法将构建好的链表头指针传递回 main 函数中的 head 变量

3) 相关代码

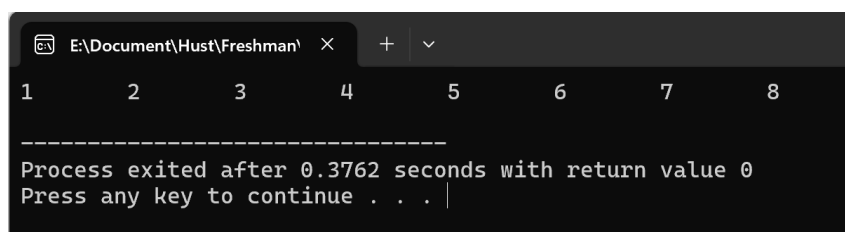
针对上述问题，我们对代码进行修改，以下是修改后的代码
更新代码 (26):

Listing 26 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      struct s_list
4      {
5          int data;
6          struct s_list *next;
7      };
8      void create_list(struct s_list **headp, int *p);
9      int main(void)
10     {
11         struct s_list *head = NULL;
12         int s[] = {1, 2, 3, 4, 5, 6, 7, 8, 0};
13         create_list(&head, s);
14         struct s_list *p = head;
15         while (p)
16         {
17             printf("%d\t", p->data);
18             p = p->next;
19         }
20         printf("\n");
21
22         return 0;
23     }
24     void create_list(struct s_list **headp, int *p)
25     {
26         struct s_list *loc_head = NULL;
27         struct s_list *tail = NULL;
```

```
28         if (*p == 0)
29         {
30             return;
31         }
32         else
33         {
34             loc_head = (struct s_list *)malloc(sizeof(struct
                    s_list));
35             loc_head->data = *p++;
36             tail = loc_head;
37
38             while (*p!= 0)
39             {
40                 tail->next = (struct s_list
                    *)malloc(sizeof(struct s_list));
41                 tail = tail->next;
42                 tail->data = *p++;
43             }
44
45             tail->next = NULL;
46             *headp = loc_head;
47         }
48     }
```

4) 执行结果



```
E:\Document\Hust\Freshman  X  +  v
1      2      3      4      5      6      7      8
-----
Process exited after 0.3762 seconds with return value 0
Press any key to continue . . .
```

图 3-1 执行示例

可以见到，通过上述 (3-1) 修正，解决了原程序中参数传递、变量使用、循环判断以及链表头指针传递等方面存在的问题，使得程序能够按照要求，根据给定的整数数组（以 0 作为结束标志）正确构建先进先出的链表，并在 main 函数中成功遍历输出链表中的节点数据。

3.2.2 7-3 源程序修改替换（二）

1) 目标内容

(2) 修改替换 `create_list` 函数, 将其建成一个后进先出的链表, 后进先出链表的头指针始终指向最后创建的结点 (链头), 后建结点指向先建结点, 先建结点始终是尾结点。

2) 分析

构建后进先出链表与先进先出链表的关键区别在于节点插入的顺序。先进先出链表是新节点依次添加到链表尾部, 就像排队一样, 先到的排在前面 (链表头), 后到的排在后面 (链表尾); 而后进先出链表则类似栈的操作, 新节点要插入到链表头部, 后加入的节点成为链表头, 这样在后续遍历输出时, 后加入的节点会先被访问到, 符合后进先出的特性。

3) 相关代码

相关代码 (27):

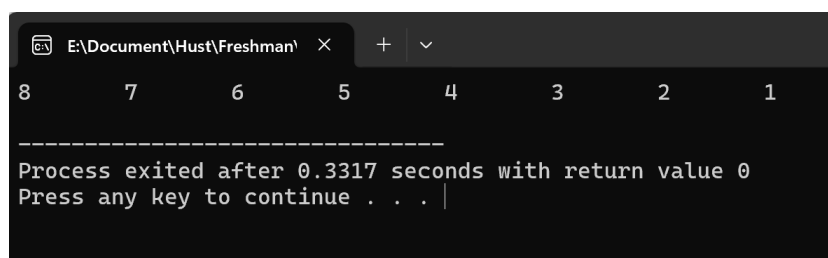
Listing 27 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3
4      struct s_list
5      {
6          int data;
7          struct s_list *next;
8      };
9
10     void create_list(struct s_list **headp, int *p);
11     int main(void)
12     {
13         struct s_list *head = NULL;
14         int s[] = {1, 2, 3, 4, 5, 6, 7, 8, 0};
15         create_list(&head, s);
16         struct s_list *p = head;
17         while (p)
18         {
19             printf("%d\t", p->data);
20             p = p->next;
21         }
```

```
22         printf("\n");
23
24         return 0;
25     }
26
27     void create_list(struct s_list **headp, int *p)
28     {
29         struct s_list *new_node;
30         if (*p == 0)
31         {
32             return;
33         }
34         else
35         {
36             while (*p != 0)
37             {
38                 new_node = (struct s_list
39                             *)malloc(sizeof(struct s_list));
40                 new_node->data = *p++;
41                 new_node->next = *headp;
42                 *headp = new_node;
43             }
44         }
```

4) 执行结果

执行结果 (3-2)



```
E:\Document\Hust\Freshman  ×  +  ▾
8      7      6      5      4      3      2      1
-----
Process exited after 0.3317 seconds with return value 0
Press any key to continue . . .
```

图 3-2 执行示例

3.3 程序设计

3.3.1 7-4 设计字段结构

1) 目标内容

设计一个字段结构 `struct bits`, 它将一个 8 位无符号字节从最低位到最高位声明为 8 个字段, 依次为 `bit0`, `bit1`, ..., `bit7`, 且 `bit0` 优先级最高。同时设计 8 个函数, 第 i 个函数以 `biti` ($i=0, 1, \dots, 7$) 为参数, 并且在函数体内输出 `biti` 的值。将 8 个函数的名字存入一个函数指针数组 `p_fun`。如果 `bit0` 为 1, 调用 `p_fun[0]` 指向的函数。如果 `struct bits` 中有多位为 1, 则根据优先级从高到低顺序依次调用函数指针数组 `p_fun` 中相应元素指向的函数。

2) 分析

我们需要设计一个 `struct bits` 结构体, 将一个 8 位无符号字节从最低位到最高位声明为 8 个字段, 依次为 `bit0`, `bit1`, ..., `bit7`, 且 `bit0` 优先级最高。同时设计 8 个函数, 每个函数以 `biti` 为参数, 并输出 `biti` 的值。将这 8 个函数的名字存入一个函数指针数组 `p_fun`, 并根据 `biti` 的值从高到低调用相应的函数。

总流程图 (3-3):

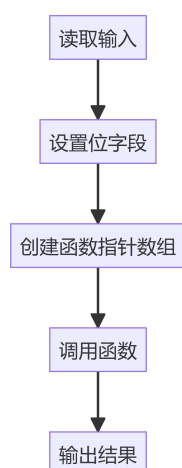


图 3-3 总流程图

思路:

对于这题主要的可分为两部分:

1. 输入处理与位提取: 先获取用户输入的整数, 然后运用位运算技巧, 依次提取该整数二进制表示下每一位的值, 并将这些值赋给结构体 `struct bits` 对应的位域

字段，完成数据的初始化准备工作，使得结构体能够准确反映输入整数的二进制位状态。

2. 函数调用逻辑构建：创建函数指针数组并初始化其元素为对应的函数地址后，通过循环和条件判断来检查结构体中位域字段的状态。按照从低位到高位（即 bit0 优先级最高，依次到 bit7）的顺序，只要发现某位的值为 1，就调用函数指针数组中对应下标的函数，同时将该位清零，保证每个置位的位对应的函数只会被调用一次，且遵循优先级顺序，直到所有置位的位对应的函数都调用完毕。

相关流程图 (3-4)：

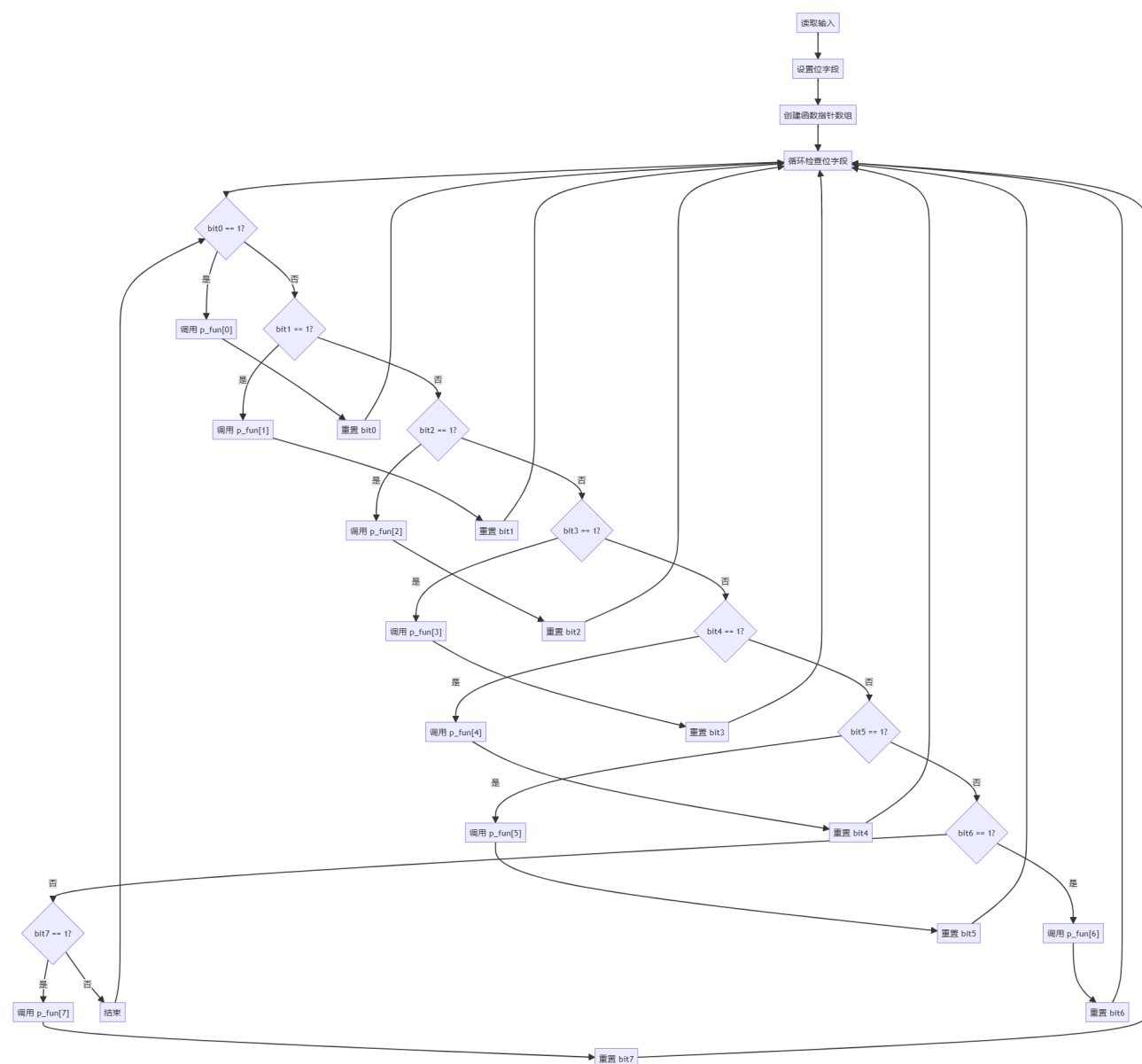


图 3-4 流程图

3) 相关代码

相关代码 (28):

Listing 28 示例

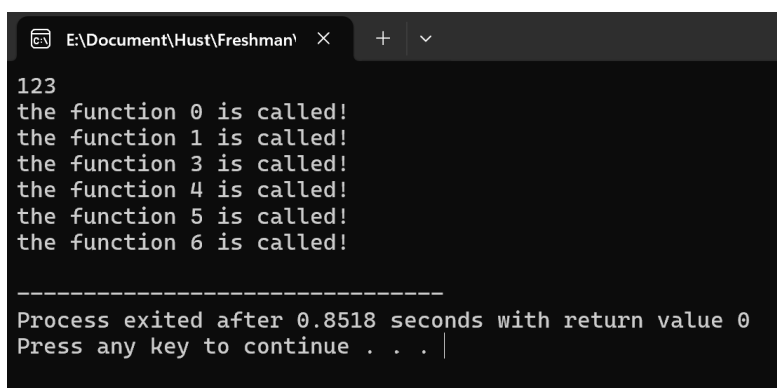
```
1      #include <stdio.h>
2      struct bits {
3          unsigned int bit0 : 1;
4          unsigned int bit1 : 1;
5          unsigned int bit2 : 1;
6          unsigned int bit3 : 1;
7          unsigned int bit4 : 1;
8          unsigned int bit5 : 1;
9          unsigned int bit6 : 1;
10         unsigned int bit7 : 1;
11     };
12     void f0(int b) {
13         printf("the function %d is called!\n", b);
14     }
15
16     void f1(int b) {
17         printf("the function %d is called!\n", b);
18     }
19
20     void f2(int b) {
21         printf("the function %d is called!\n", b);
22     }
23
24     void f3(int b) {
25         printf("the function %d is called!\n", b);
26     }
27
28     void f4(int b) {
29         printf("the function %d is called!\n", b);
30     }
31
32     void f5(int b) {
33         printf("the function %d is called!\n", b);
34     }
35
36     void f6(int b) {
```

```
37         printf("the function %d is called!\n", b);
38     }
39
40     void f7(int b) {
41         printf("the function %d is called!\n", b);
42     }
43     int main() {
44         struct bits byte;
45         int num;
46         scanf("%d", &num);
47         byte.bit0 = num & 1;
48         byte.bit1 = (num >> 1) & 1;
49         byte.bit2 = (num >> 2) & 1;
50         byte.bit3 = (num >> 3) & 1;
51         byte.bit4 = (num >> 4) & 1;
52         byte.bit5 = (num >> 5) & 1;
53         byte.bit6 = (num >> 6) & 1;
54         byte.bit7 = (num >> 7) & 1;
55         void (*p_fun[8])(int) = {f0, f1, f2, f3, f4, f5, f6, f7};
56         for (int i = 0; i < 8; i++) {
57             if (byte.bit0 + byte.bit1 + byte.bit2 + byte.bit3 +
58                 byte.bit4 + byte.bit5 + byte.bit6 + byte.bit7 >
59                 0) {
60                 int index = 0;
61                 while (byte.bit0 + byte.bit1 + byte.bit2 +
62                     byte.bit3 + byte.bit4 + byte.bit5 +
63                     byte.bit6 + byte.bit7 > 0) {
64                     if (byte.bit0 == 1) {
65                         p_fun[0](0);
66                         byte.bit0 = 0;
67                     }
68                     else if (byte.bit1 == 1) {
69                         p_fun[1](1);
70                         byte.bit1 = 0;
71                     }
72                     else if (byte.bit2 == 1) {
73                         p_fun[2](2);
74                         byte.bit2 = 0;
75                     }
76                     else if (byte.bit3 == 1) {
77                         p_fun[3](3);
78                         byte.bit3 = 0;
79                     }
80                     else if (byte.bit4 == 1) {
81                         p_fun[4](4);
82                         byte.bit4 = 0;
83                     }
84                     else if (byte.bit5 == 1) {
85                         p_fun[5](5);
86                         byte.bit5 = 0;
87                     }
88                     else if (byte.bit6 == 1) {
89                         p_fun[6](6);
90                         byte.bit6 = 0;
91                     }
92                     else if (byte.bit7 == 1) {
93                         p_fun[7](7);
94                         byte.bit7 = 0;
95                     }
96                 }
97             }
98         }
```

```
74         byte.bit3 = 0;
75     }
76     else if (byte.bit4 == 1) {
77         p_fun[4](4);
78         byte.bit4 = 0;
79     }
80     else if (byte.bit5 == 1) {
81         p_fun[5](5);
82         byte.bit5 = 0;
83     }
84     else if (byte.bit6 == 1) {
85         p_fun[6](6);
86         byte.bit6 = 0;
87     }
88     else if (byte.bit7 == 1) {
89         p_fun[7](7);
90         byte.bit7 = 0;
91     }
92 }
93 break;
94 }
95 }
96 return 0;
97 }
```

4) 执行结果

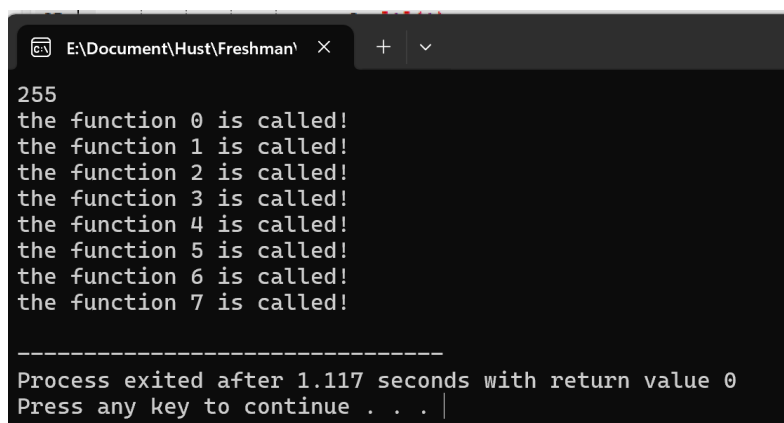
执行结果 (3-53-6)



```
E:\Document\Hust\Freshman' x + v
123
the function 0 is called!
the function 1 is called!
the function 3 is called!
the function 4 is called!
the function 5 is called!
the function 6 is called!

-----
Process exited after 0.8518 seconds with return value 0
Press any key to continue . . . |
```

图 3-5 执行示例



```
E:\Document\Hust\Freshman\  X  +  v
255
the function 0 is called!
the function 1 is called!
the function 2 is called!
the function 3 is called!
the function 4 is called!
the function 5 is called!
the function 6 is called!
the function 7 is called!

-----
Process exited after 1.117 seconds with return value 0
Press any key to continue . . . |
```

图 3-6 执行示例

可见，通过前面的分析，设计，程序能够正确地解析输入的 8 位无符号整数，并根据位字段的值从高到低调用相应的函数。

3.3.2 7-5 班级成绩单

1) 目标内容

用单向链表建立一张班级成绩单，包括每个学生的学号、姓名、英语、高等数学、普通物理、C 语言程序设计 4 门课程的成绩。实现以下功能，并提供菜单选项：

0. 退出

1. 输入每个学生的各项信息

2. 输出每个学生的各项信息

3. 修改指定学生的指定数据项的内容：

1. 修改英语成绩

2. 修改高等数学成绩

3. 修改普通物理成绩

4. 修改 C 语言成绩

4. 统计每个学生的平均成绩（保留 2 位小数）

5. 输出各位学生的学号、姓名、4 门课程的总成绩和平均成绩

2) 分析

这题的目标内容和数组的实验题几许相似，我们需要利用结构体和模块化的设计思路去进行分块设计。有助于我们去一步步完成这题的设计。

总流程图 (3-15)：

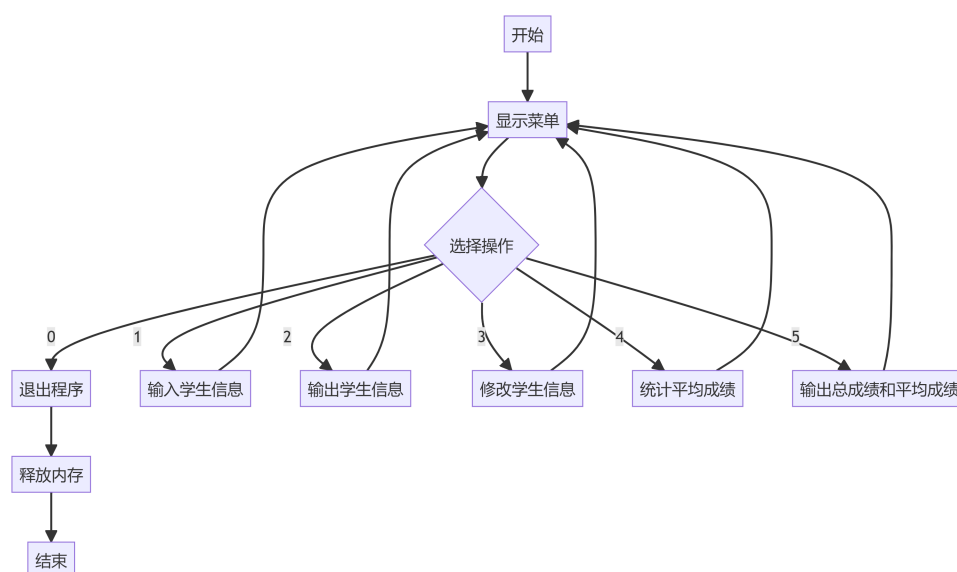


图 3-7 总流程图

設計思路：

选用单向链表来存储班级学生的成绩单信息，是因为班级学生数量不确定，链表这种动态数据结构可以方便地根据需要添加或删除节点，灵活地管理不同数量的学生数据。每个链表节点对应一个学生，节点结构体中包含了存储学生各项信息（学号、姓名、四门课程成绩以及平均成绩）的成员变量，以及一个指向下一个节点的指针，通过指针将各个节点串联起来形成完整的成绩单链表。通过构建函数创建单个学生节点并初始化各项成员变量（含成绩赋值与平均成绩计算），构建新函数将新节点添加到链表中，处理链表为空和不为空的添加逻辑，以此构建整个班级成绩单链表；再设计函数按不同需求输出学生详细信息、用自定义的函数输出平均成绩、函数输出学号、姓名、总成绩及平均成绩等多个输出函数，它们都通过遍历链表展示相应信息方便用户查看不同格式成绩单内容；再设计函数可遍历链表查找匹配学号的节点，依据用户所选课程选项更新成绩并重新算平均成绩来实现修改指定学生指定课程成绩功能，保证数据准确一致；在 `main` 函数里利用无限循环和 `switch` 语句构建菜单驱动的程序框架，按用户输入选项调用相应功能函数实现交互及各功能调度，且退出程序选项中考虑释放链表内存资源以避免内存泄漏问题。相关流程图 (3-8)：

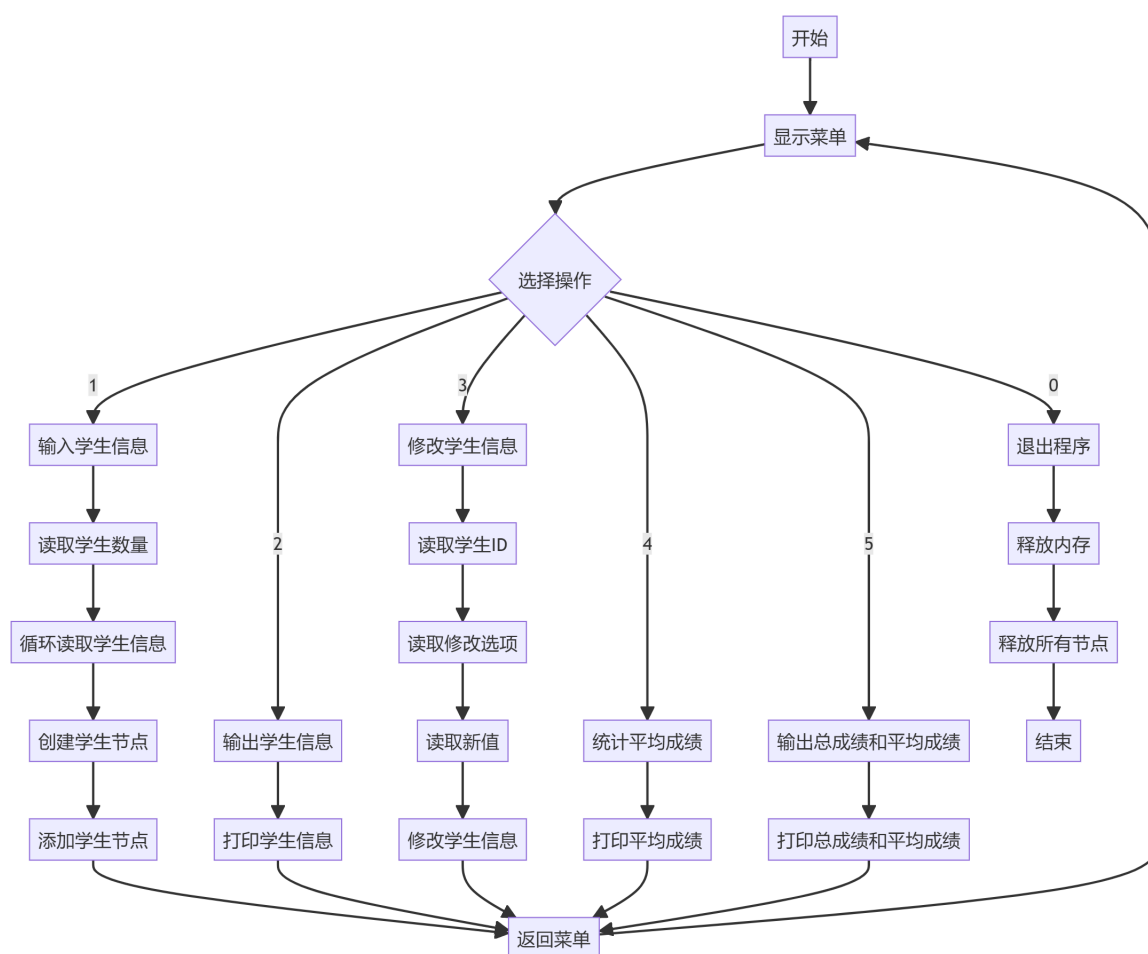


图 3-8 流程图

3) 相关代码

相关代码 (29):

Listing 29 示例

```

1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4      #include <math.h>
5      #define NAME_LEN 50
6      typedef struct Student {
7          char id[20];
8          char name[NAME_LEN];
9          float english;
10         float math;
11         float physics;
    
```

华中科技大学课程实验报告

```
12         float c_programming;
13         float average;
14         struct Student* next;
15     } Student;
16     Student* createNode(char* id, char* name, float english,
17         float math, float physics, float c_programming) {
18         Student* newNode = (Student*)malloc(sizeof(Student));
19         if (newNode == NULL) {
20             exit(1);
21         }
22         strcpy(newNode->id, id);
23         strcpy(newNode->name, name);
24         newNode->english = english;
25         newNode->math = math;
26         newNode->physics = physics;
27         newNode->c_programming = c_programming;
28         newNode->average = (english + math + physics +
29             c_programming) / 4.0;
30         newNode->next = NULL;
31         return newNode;
32     }
33     void addStudent(Student** head, char* id, char* name, float
34         english, float math, float physics, float
35         c_programming) {
36         Student* newNode = createNode(id, name, english,
37             math, physics, c_programming);
38         if (*head == NULL) {
39             *head = newNode;
40         } else {
41             Student* temp = *head;
42             while (temp->next != NULL) {
43                 temp = temp->next;
44             }
45             temp->next = newNode;
46         }
47     }
48     void printStudents(Student* head, int isAveragePrinted) {
49         Student* temp = head;
50         while (temp != NULL) {
51             printf("%s %s ", temp->id, temp->name);
52             if (isAveragePrinted) {
```

```
48         printf("%.2f ", temp->english);
49         printf("%.2f ", temp->math);
50         printf("%.2f ", temp->physics);
51         printf("%.2f\n", temp->c_programming);
52     } else {
53         if (floor(temp->english) ==
54             temp->english) {
55             printf("%d ",
56                 (int)temp->english);
57         } else {
58             printf("%.2f ", temp->english);
59         }
60         if (floor(temp->math) == temp->math) {
61             printf("%d ", (int)temp->math);
62         } else {
63             printf("%.2f ", temp->math);
64         }
65         if (floor(temp->physics) ==
66             temp->physics) {
67             printf("%d ",
68                 (int)temp->physics);
69         } else {
70             printf("%.2f ", temp->physics);
71         }
72         if (floor(temp->c_programming) ==
73             temp->c_programming) {
74             printf("%d\n",
75                 (int)temp->c_programming);
76         } else {
77             printf("%.2f\n",
78                 temp->c_programming);
79         }
80     }
81     temp = temp->next;
82 }
83
84 void modifyStudent(Student* head, char* id, int choice,
85     float newValue) {
86     Student* temp = head;
87     while (temp != NULL) {
88         if (strcmp(temp->id, id) == 0) {
```

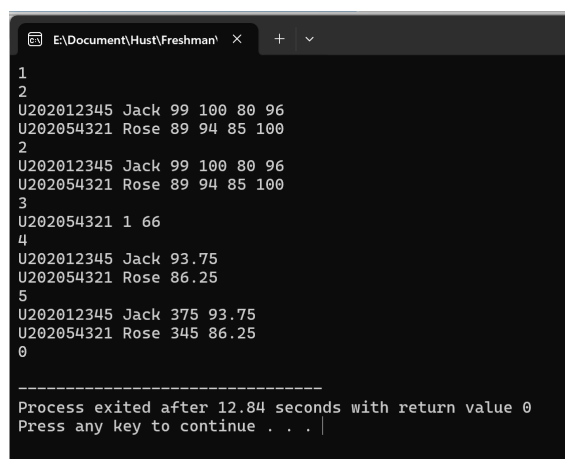
```
81         switch (choice) {
82             case 1:
83                 temp->english = newValue;
84                 break;
85             case 2:
86                 temp->math = newValue;
87                 break;
88             case 3:
89                 temp->physics = newValue;
90                 break;
91             case 4:
92                 temp->c_programming = newValue;
93                 break;
94         }
95         temp->average = (temp->english +
96                         temp->math + temp->physics +
97                         temp->c_programming) / 4.0;
98         return;
99     }
100 }
101 void printAverages(Student* head) {
102     Student* temp = head;
103     while (temp != NULL) {
104         printf("%s %s ", temp->id, temp->name);
105         printf("%.2f\n", temp->average);
106         temp = temp->next;
107     }
108 }
109 void printTotalAndAverage(Student* head) {
110     Student* temp = head;
111     while (temp != NULL) {
112         float total = temp->english + temp->math +
113                     temp->physics + temp->c_programming;
114         printf("%s %s %d %.2f\n", temp->id,
115                temp->name, (int)total, temp->average);
116         temp = temp->next;
117     }
118 }
119 int main() {
```

```
118         Student* head = NULL;
119         int choice, n;
120         char id[20], name[NAME_LEN];
121         float english, math, physics, c_programming;
122         char modifyId[20];
123         int modifyChoice;
124         float newValue;
125         int isAveragePrinted = 0;
126         while (1) {
127             scanf("%d", &choice);
128
129             switch (choice) {
130                 case 0:
131                     while (head != NULL) {
132                         Student* temp = head;
133                         head = head->next;
134                         free(temp);
135                     }
136                     exit(0);
137                 case 1:
138                     scanf("%d", &n);
139                     for (int i = 0; i < n; i++) {
140                         scanf("%s %s %f %f %f %f", id,
141                             name, &english, &math,
142                             &physics, &c_programming);
143                         addStudent(&head, id, name,
144                             english, math, physics,
145                             c_programming);
146                     }
147                     break;
148                 case 2:
149                     printStudents(head, isAveragePrinted);
150                     break;
151                 case 3:
152                     scanf("%s %d %f", modifyId,
153                         &modifyChoice, &newValue);
154                     modifyStudent(head, modifyId,
155                         modifyChoice, newValue);
156                     break;
157                 case 4:
158                     printAverages(head);
```

```
153         isAveragePrinted = 1;
154         break;
155     case 5:
156         printTotalAndAverage(head);
157         break;
158     }
159 }
160 return 0;
161 }
```

4) 执行结果

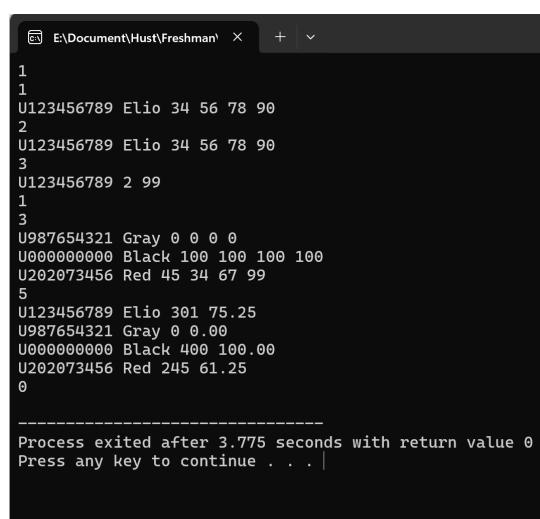
执行结果 (3-93-10):



```
E:\Document\Hust\Freshman\ x + v
1
2
U202012345 Jack 99 100 80 96
U202054321 Rose 89 94 85 100
2
U202012345 Jack 99 100 80 96
U202054321 Rose 89 94 85 100
3
U202054321 1 66
4
U202012345 Jack 93.75
U202054321 Rose 86.25
5
U202012345 Jack 375 93.75
U202054321 Rose 345 86.25
0

-----
Process exited after 12.84 seconds with return value 0
Press any key to continue . . . |
```

图 3-9 执行示例



```
E:\Document\Hust\Freshman\ x + v
1
1
U123456789 Elio 34 56 78 90
2
U123456789 Elio 34 56 78 90
3
U123456789 2 99
1
3
U987654321 Gray 0 0 0 0
U000000000 Black 100 100 100 100
U202073456 Red 45 34 67 99
5
U123456789 Elio 301 75.25
U987654321 Gray 0 0.00
U000000000 Black 400 100.00
U202073456 Red 245 61.25
0

-----
Process exited after 3.775 seconds with return value 0
Press any key to continue . . . |
```

图 3-10 执行示例

通过输入和处理学生的数据，能够见到本代码在最后是能够顺利输出的，实

现了本题的要求，下面把与这题有相关的 7-6 和 7-8 题目也一并说明。

3.3.3 7-6 成绩排序（一）

1) 目标内容

对程序设计的第二题增加按照平均成绩进行升序排序的函数，写出用交换结点数据域的方法升序排序的函数，排序可用选择法或冒泡法。

2) 分析

在 7-5 原有的基础上加入排序的算法，基本上大致的流程结构不用改变多少，主要是在模块化的地方加入新的排序算法，则能使本题成立。总流程图 (3-15)：

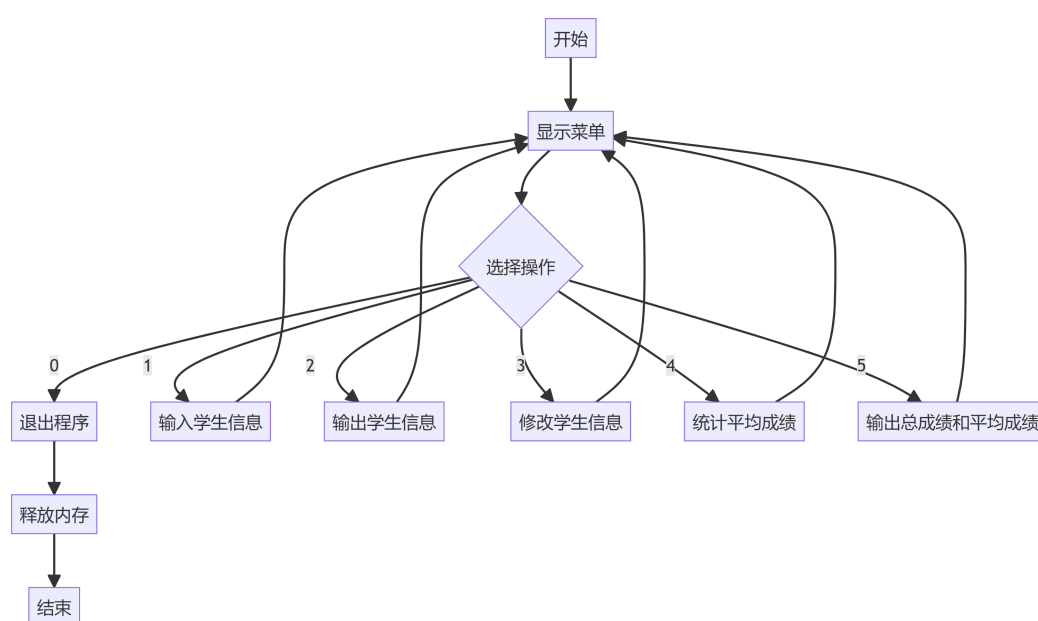


图 3-11 总流程图

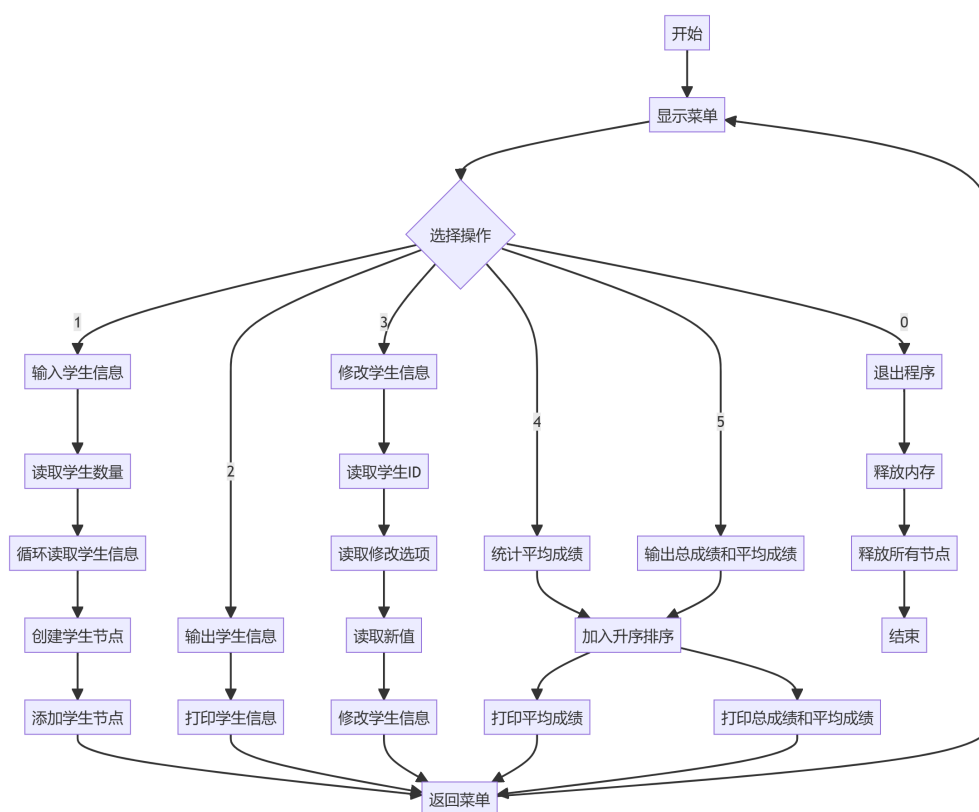


图 3-12 流程图

3) 相关代码

相关代码 (30):

Listing 30 示例

```

1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4      #include <math.h>
5      #define NAME_LEN 50
6      typedef struct Student {
7          char id[20];
8          char name[NAME_LEN];
9          float english;
10         float math;
11         float physics;
12         float c_programming;
13         float average;
14         struct Student *next;
15     } Student;
    
```

```
16      Student *createNode(char *id, char *name, float english,
17                          float math, float physics, float c_programming);
18      void addStudent(Student **head, char *id, char *name, float
19                      english, float math, float physics, float
20                      c_programming);
21      void printStudents(Student *head, int isAveragePrinted);
22      void modifyStudent(Student *head, char *id, int choice,
23                          float newValue);
24      void sortStudentsByAverage(Student **head);
25      void printAverages(Student *head);
26      void printTotalAndAverage(Student *head);
27      Student *createNode(char *id, char *name, float english,
28                          float math, float physics, float c_programming) {
29          Student *newNode = (Student
30                              *)malloc(sizeof(Student));
31          if (newNode == NULL) {
32              exit(1);
33          }
34          strcpy(newNode->id, id);
35          strcpy(newNode->name, name);
36          newNode->english = english;
37          newNode->math = math;
38          newNode->physics = physics;
39          newNode->c_programming = c_programming;
40          newNode->average = (english + math + physics +
41                              c_programming) / 4.0;
42          newNode->next = NULL;
43          return newNode;
44      }
45      void addStudent(Student **head, char *id, char *name, float
46                      english, float math, float physics, float
47                      c_programming) {
48          Student *newNode = createNode(id, name, english,
49                                          math, physics, c_programming);
50          if (*head == NULL) {
51              *head = newNode;
52          }
53          else {
54              Student *temp = *head;
55              while (temp->next != NULL) {
56                  temp = temp->next;
```

```
47         }
48         temp->next = newNode;
49     }
50 }
51 void printStudents(Student *head, int isAveragePrinted) {
52     Student *temp = head;
53     while (temp != NULL) {
54         printf("%s %s ", temp->id, temp->name);
55         if (isAveragePrinted) {
56             printf("%.2f ", temp->english);
57             printf("%.2f ", temp->math);
58             printf("%.2f ", temp->physics);
59             printf("%.2f\n", temp->c_programming);
60         }
61         else {
62             if ((int)temp->english ==
63                 temp->english) {
64                 printf("%d ",
65                     (int)temp->english);
66             }
67             else {
68                 printf("%.2f ", temp->english);
69             }
70             if ((int)temp->math == temp->math) {
71                 printf("%d ", (int)temp->math);
72             }
73             else {
74                 printf("%.2f ", temp->math);
75             }
76             if ((int)temp->physics ==
77                 temp->physics) {
78                 printf("%d ",
79                     (int)temp->physics);
80             }
81             else {
82                 printf("%.2f ", temp->physics);
83             }
84             if ((int)temp->c_programming ==
85                 temp->c_programming) {
86                 printf("%d\n",
87                     (int)temp->c_programming);
```

华中科技大学课程实验报告

```
82         }
83         else {
84             printf("%.2f\n",
85                     temp->c_programming);
86         }
87         temp = temp->next;
88     }
89 }
90 void modifyStudent(Student *head, char *id, int choice,
91                   float newValue) {
92     Student *temp = head;
93     while (temp != NULL) {
94         if (strcmp(temp->id, id) == 0) {
95             switch (choice) {
96                 case 1:
97                     temp->english = newValue;
98                     break;
99                 case 2:
100                     temp->math = newValue;
101                     break;
102                 case 3:
103                     temp->physics = newValue;
104                     break;
105                 case 4:
106                     temp->c_programming = newValue;
107                     break;
108             }
109             temp->average = (temp->english +
110                             temp->math + temp->physics +
111                             temp->c_programming) / 4.0;
112             return;
113         }
114         temp = temp->next;
115     }
116 }
117 void sortStudentsByAverage(Student **head) {
118     if (*head == NULL || (*head)->next == NULL) {
119         return;
120     }
121     Student *p;
```

```
119         Student *last = NULL;
120         int swapped;
121         do {
122             swapped = 0;
123             p = *head;
124             while (p->next!= last) {
125                 if (p->average > p->next->average) {
126                     Student temp;
127                     strcpy(temp.id, p->id);
128                     strcpy(p->id, p->next->id);
129                     strcpy(p->next->id, temp.id);
130                     strcpy(temp.name, p->name);
131                     strcpy(p->name, p->next->name);
132                     strcpy(p->next->name, temp.name);
133                     temp.english = p->english;
134                     p->english = p->next->english;
135                     p->next->english = temp.english;
136                     temp.math = p->math;
137                     p->math = p->next->math;
138                     p->next->math = temp.math;
139                     temp.physics = p->physics;
140                     p->physics = p->next->physics;
141                     p->next->physics = temp.physics;
142                     temp.c_programming =
143                         p->c_programming;
144                     p->c_programming =
145                         p->next->c_programming;
146                     p->next->c_programming =
147                         temp.c_programming;
148                     temp.average = p->average;
149                     p->average = p->next->average;
150                     p->next->average = temp.average;
151                     swapped = 1;
152                 }
153                 p = p->next;
154             }
155             last = p;
156         } while (swapped);
157     }
158
159     void printAverages(Student *head) {
160         Student *temp = head;
```

```
157         while (temp!= NULL) {
158             printf("%s %s %.2f\n", temp->id, temp->name,
159                 temp->average);
160             temp = temp->next;
161         }
162     }
163     void printTotalAndAverage(Student *head) {
164         Student *temp = head;
165         while (temp!= NULL) {
166             float total = temp->english + temp->math +
167                 temp->physics + temp->c_programming;
168             printf("%s %s %d %.2f\n", temp->id,
169                 temp->name, (int)total, temp->average);
170             temp = temp->next;
171         }
172     }
173     int main() {
174         Student *head = NULL;
175         int choice, n;
176         char id[20], name[NAME_LEN];
177         float english, math, physics, c_programming;
178         char modifyId[20];
179         int modifyChoice;
180         float newValue;
181         int isAveragePrinted = 0;
182         while (1) {
183             scanf("%d", &choice);
184             switch (choice) {
185                 case 0:
186                     while (head!= NULL) {
187                         Student *temp = head;
188                         head = head->next;
189                         free(temp);
190                     }
191                     exit(0);
192                 case 1:
193                     scanf("%d", &n);
194                     for (int i = 0; i < n; i++) {
195                         scanf("%s %s %f %f %f %f", id,
196                             name, &english, &math,
197                             &physics, &c_programming);
```

```
193             addStudent(&head, id, name,
194                        english, math, physics,
195                        c_programming);
196         }
197         break;
198         case 2:
199             sortStudentsByAverage(&head);
200             printStudents(head, isAveragePrinted);
201             break;
202         case 3:
203             scanf("%s %d %f", modifyId,
204                  &modifyChoice, &newValue);
205             modifyStudent(head, modifyId,
206                          modifyChoice, newValue);
207             break;
208         case 4:
209             sortStudentsByAverage(&head);
210             printAverages(head);
211             isAveragePrinted = 1;
212             break;
213         case 5:
214             sortStudentsByAverage(&head);
215             printTotalAndAverage(head);
216             break;
217     }
218 }
219 return 0;
220 }
```

4) 执行结果

执行结果 (3-133-14):

```
E:\Document\Hust\Freshman' x + v
1
2
U202054321 Rose 89 94 85 100
U202012345 Jack 99 100 80 96
2
U202054321 Rose 89 94 85 100
U202012345 Jack 99 100 80 96
3
U202054321 1 66
4
U202054321 Rose 86.25
U202012345 Jack 93.75
5
U202054321 Rose 345 86.25
U202012345 Jack 375 93.75
0

-----
Process exited after 14.61 seconds with return value 0
Press any key to continue . . . |
```

图 3-13 执行示例

```
E:\Document\Hust\Freshman' x + v
1
4
U202054321 Rose 89 94 85 100
U202056789 Tom 12 34 56 78
U202012345 Jack 99 100 80 96
U202098765 Jerry 98 76 54 32
2
U202056789 Tom 12 34 56 78
U202098765 Jerry 98 76 54 32
U202054321 Rose 89 94 85 100
U202012345 Jack 99 100 80 96
3
U202054321 1 66
4
U202056789 Tom 45.00
U202098765 Jerry 65.00
U202054321 Rose 86.25
U202012345 Jack 93.75
5
U202056789 Tom 180 45.00
U202098765 Jerry 260 65.00
U202054321 Rose 345 86.25
U202012345 Jack 375 93.75
0

-----
Process exited after 2.085 seconds with return value 0
Press any key to continue . . . |
```

图 3-14 执行示例

3.3.4 7-8 成绩排序（二）

1) 目标内容

对于程序设计第三题，进一步写出用交换结点指针域的方法升序排序的函数。

2) 分析

采用交换节点指针域的方式来进行链表排序，同样可以选择如选择排序或冒泡排序的基本思想来实现升序排列。这里以选择排序为例进行说明，整体思路是每次遍历链表未排序部分，找到平均成绩最小的节点对应的指针，然后通过调整指针域，也就是改变节点之间的链接关系，将这个最小平均成绩的节点移动到已排序部分的末尾，然后经过多轮这样的操作，最终使得整个链表按照学生平均成绩升序排列。总流程图 (3-15)：

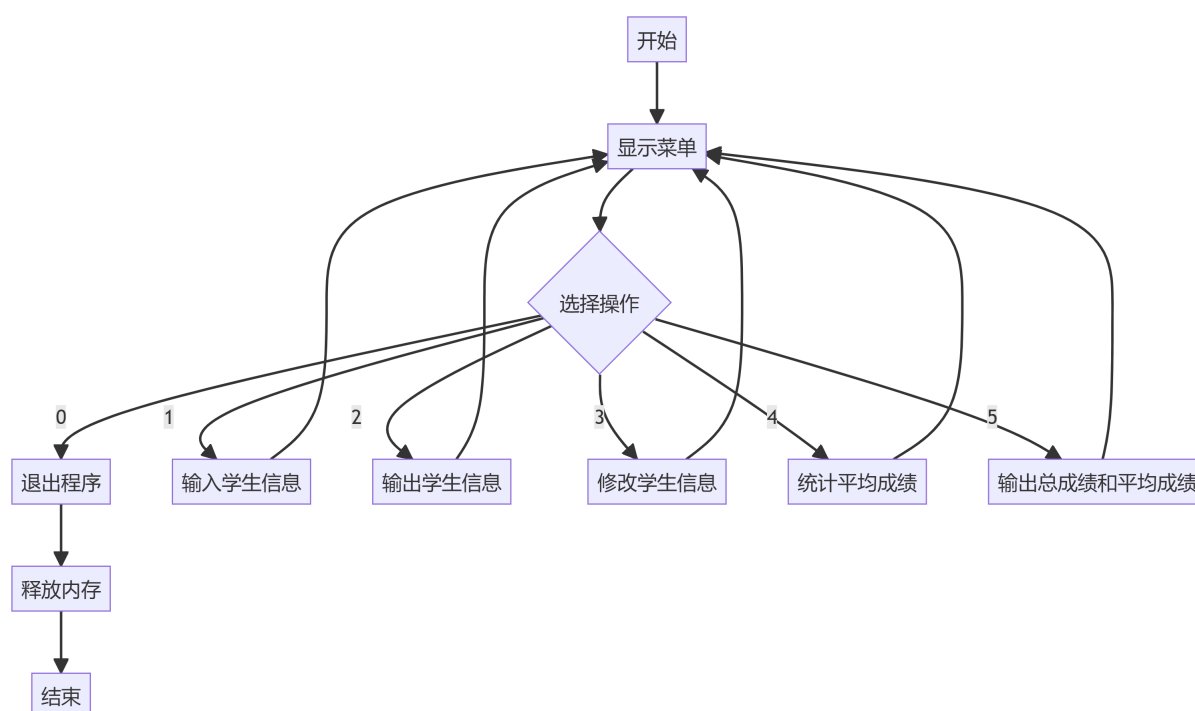


图 3-15 总流程图

思路：

我们可以通过两层嵌套的循环来遍历链表，外层循环用于控制每一轮排序操作，确定当前已排序部分的末尾位置（也就是下一个要插入最小平均成绩节点的位置）；内层循环则从外层循环当前位置的下一个节点开始，遍历未排序部分的所

有节点，比较各节点的平均成绩，记录下平均成绩最小的节点的指针。

而当找到未排序部分中平均成绩最小的节点指针后，需要调整指针域来改变节点顺序。具体来说，要将这个最小平均成绩节点从原来的位置“脱离”出来，并插入到当前已排序部分的末尾（即当前未排序部分的开头）。这涉及到修改相关节点的 `next` 指针，使得链表的链接关系按照平均成绩升序重新排列，而节点本身存储的学生数据（学号、姓名、各门课程成绩等）并没有进行交换，只是改变了链表中节点的顺序。

相關流程图 (3-16)

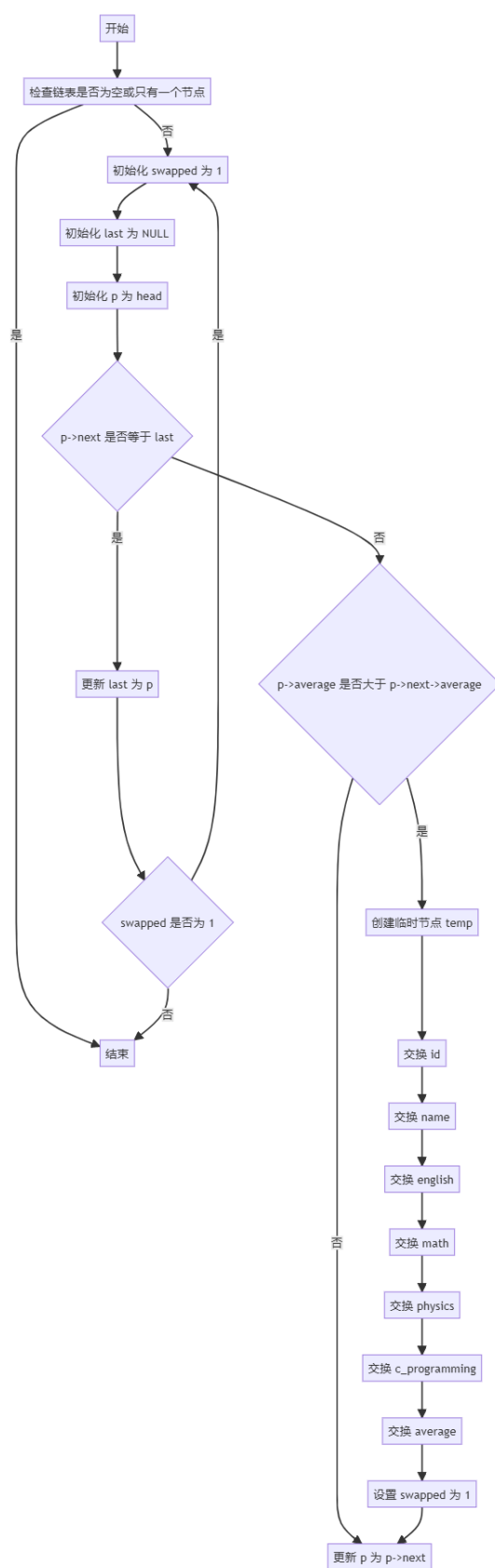


图 3-16 流程图

3) 相关代码

Listing 31 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4      #include <math.h>
5      #define NAME_LEN 50
6      typedef struct Student {
7          char id[20];
8          char name[NAME_LEN];
9          float english;
10         float math;
11         float physics;
12         float c_programming;
13         float average;
14         struct Student *next;
15     } Student;
16     Student *createNode(char *id, char *name, float english,
17         float math, float physics, float c_programming);
18     void addStudent(Student **head, char *id, char *name, float
19         english, float math, float physics, float
20         c_programming);
21     void printStudents(Student *head, int isAveragePrinted);
22     void modifyStudent(Student *head, char *id, int choice,
23         float newValue);
24     void sortStudentsByAverage(Student **head);
25     void printAverages(Student *head);
26     void printTotalAndAverage(Student *head);
27     Student *createNode(char *id, char *name, float english,
28         float math, float physics, float c_programming) {
29         Student *newNode = (Student
30             *)malloc(sizeof(Student));
31         if (newNode == NULL) {
32             exit(1);
33         }
34         strcpy(newNode->id, id);
35         strcpy(newNode->name, name);
36         newNode->english = english;
37         newNode->math = math;
38         newNode->physics = physics;
```

```
33         newNode->c_programming = c_programming;
34         newNode->average = (english + math + physics +
35                             c_programming) / 4.0;
36         newNode->next = NULL;
37         return newNode;
38     }
39     void addStudent(Student **head, char *id, char *name, float
40                     english, float math, float physics, float
41                     c_programming) {
42         Student *newNode = createNode(id, name, english,
43                                       math, physics, c_programming);
44         if (*head == NULL) {
45             *head = newNode;
46         }
47         else {
48             Student *temp = *head;
49             while (temp->next != NULL) {
50                 temp = temp->next;
51             }
52             temp->next = newNode;
53         }
54     }
55     void printStudents(Student *head, int isAveragePrinted) {
56         Student *temp = head;
57         while (temp != NULL) {
58             printf("%s %s ", temp->id, temp->name);
59             if (isAveragePrinted) {
60                 printf("%.2f ", temp->english);
61                 printf("%.2f ", temp->math);
62                 printf("%.2f ", temp->physics);
63                 printf("%.2f\n", temp->c_programming);
64             }
65             else {
66                 if ((int)temp->english ==
67                     temp->english) {
68                     printf("%d ",
69                             (int)temp->english);
70                 }
71                 else {
72                     printf("%.2f ", temp->english);
73                 }
74             }
75         }
76     }
```

```
68         if ((int)temp->math == temp->math) {
69             printf("%d ", (int)temp->math);
70         }
71         else {
72             printf("%.2f ", temp->math);
73         }
74         if ((int)temp->physics ==
75             temp->physics) {
76             printf("%d ",
77                 (int)temp->physics);
78         }
79         else {
80             printf("%.2f ", temp->physics);
81         }
82         if ((int)temp->c_programming ==
83             temp->c_programming) {
84             printf("%d\n",
85                 (int)temp->c_programming);
86         }
87         else {
88             printf("%.2f\n",
89                 temp->c_programming);
90         }
91     }
92     temp = temp->next;
93 }
94
95 void modifyStudent(Student *head, char *id, int choice,
96     float newValue) {
97     Student *temp = head;
98     while (temp != NULL) {
99         if (strcmp(temp->id, id) == 0) {
100             switch (choice) {
101                 case 1:
102                     temp->english = newValue;
103                     break;
104                 case 2:
105                     temp->math = newValue;
106                     break;
107                 case 3:
108                     temp->physics = newValue;
```

```
103             break;
104             case 4:
105                 temp->c_programming = newValue;
106                 break;
107         }
108         temp->average = (temp->english +
109             temp->math + temp->physics +
110             temp->c_programming) / 4.0;
111         return;
112     }
113     temp = temp->next;
114 }
115
116 void sortStudentsByAverage(Student **head) {
117     if (*head == NULL || (*head)->next == NULL) {
118         return;
119     }
120     Student *p;
121     Student *last = NULL;
122     int swapped;
123     do {
124         swapped = 0;
125         p = *head;
126         while (p->next != last) {
127             if (p->average > p->next->average) {
128                 Student temp;
129                 strcpy(temp.id, p->id);
130                 strcpy(p->id, p->next->id);
131                 strcpy(p->next->id, temp.id);
132                 strcpy(temp.name, p->name);
133                 strcpy(p->name, p->next->name);
134                 strcpy(p->next->name, temp.name);
135                 temp.english = p->english;
136                 p->english = p->next->english;
137                 p->next->english = temp.english;
138                 temp.math = p->math;
139                 p->math = p->next->math;
140                 p->next->math = temp.math;
141                 temp.physics = p->physics;
142                 p->physics = p->next->physics;
143                 p->next->physics = temp.physics;
```

```
142         temp.c_programming =
143             p->c_programming;
144         p->c_programming =
145             p->next->c_programming;
146         p->next->c_programming =
147             temp.c_programming;
148         temp.average = p->average;
149         p->average = p->next->average;
150         p->next->average = temp.average;
151         swapped = 1;
152     }
153     p = p->next;
154 }
155 last = p;
156 } while (swapped);
157 }
158 void printAverages(Student *head) {
159     Student *temp = head;
160     while (temp != NULL) {
161         printf("%s %s %.2f\n", temp->id, temp->name,
162             temp->average);
163         temp = temp->next;
164     }
165 }
166 void printTotalAndAverage(Student *head) {
167     Student *temp = head;
168     while (temp != NULL) {
169         float total = temp->english + temp->math +
170             temp->physics + temp->c_programming;
171         printf("%s %s %d %.2f\n", temp->id,
172             temp->name, (int)total, temp->average);
173         temp = temp->next;
174     }
175 }
176 int main() {
177     Student *head = NULL;
178     int choice, n;
179     char id[20], name[NAME_LEN];
180     float english, math, physics, c_programming;
181     char modifyId[20];
182     int modifyChoice;
```

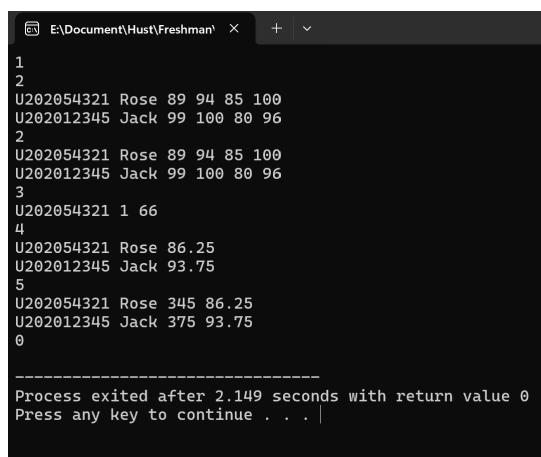
华中科技大学课程实验报告

```
177         float newValue;
178         int isAveragePrinted = 0;
179         while (1) {
180             scanf("%d", &choice);
181
182             switch (choice) {
183                 case 0:
184                     while (head!= NULL) {
185                         Student *temp = head;
186                         head = head->next;
187                         free(temp);
188                     }
189                     exit(0);
190                 case 1:
191                     scanf("%d", &n);
192                     for (int i = 0; i < n; i++) {
193                         scanf("%s %s %f %f %f %f", id,
194                             name, &english, &math,
195                             &physics, &c_programming);
196                         addStudent(&head, id, name,
197                             english, math, physics,
198                             c_programming);
199                     }
200                     break;
201                 case 2:
202                     sortStudentsByAverage(&head);
203                     printStudents(head, isAveragePrinted);
204                     break;
205                 case 3:
206                     scanf("%s %d %f", modifyId,
207                         &modifyChoice, &newValue);
208                     modifyStudent(head, modifyId,
209                         modifyChoice, newValue);
210                     break;
211                 case 4:
212                     sortStudentsByAverage(&head);
213                     printAverages(head);
214                     isAveragePrinted = 1;
215                     break;
216                 case 5:
217                     sortStudentsByAverage(&head);
```

```
212         printTotalAndAverage(head);
213         break;
214     }
215 }
216 return 0;
217 }
```

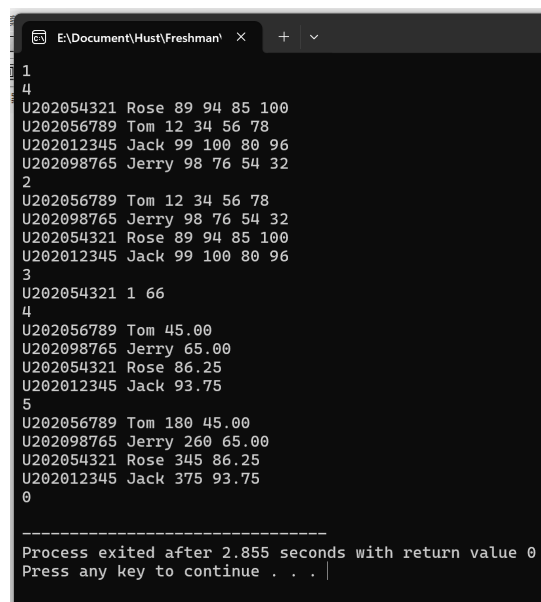
4) 执行结果

执行结果 (??3-173-17)



```
E:\Document\Hust\Freshman' x + v
1
2
U202054321 Rose 89 94 85 100
U202012345 Jack 99 100 80 96
2
U202054321 Rose 89 94 85 100
U202012345 Jack 99 100 80 96
3
U202054321 1 66
4
U202054321 Rose 86.25
U202012345 Jack 93.75
5
U202054321 Rose 345 86.25
U202012345 Jack 375 93.75
0
-----
Process exited after 2.149 seconds with return value 0
Press any key to continue . . . |
```

图 3-17 执行示例



```
E:\Document\Hust\Freshman' x + v
1
4
U202054321 Rose 89 94 85 100
U202056789 Tom 12 34 56 78
U202012345 Jack 99 100 80 96
U202098765 Jerry 98 76 54 32
2
U202056789 Tom 12 34 56 78
U202098765 Jerry 98 76 54 32
U202054321 Rose 89 94 85 100
U202012345 Jack 99 100 80 96
3
U202054321 1 66
4
U202056789 Tom 45.00
U202098765 Jerry 65.00
U202054321 Rose 86.25
U202012345 Jack 93.75
5
U202056789 Tom 180 45.00
U202098765 Jerry 260 65.00
U202054321 Rose 345 86.25
U202012345 Jack 375 93.75
0
-----
Process exited after 2.855 seconds with return value 0
Press any key to continue . . . |
```

图 3-18 执行示例

3.3.5 7-7 回文字符串

1) 目标内容

回文字符串是正读和反读都相同的字符串，例如“abba”和“abcba”是回文字符串。设计程序判断输入的任意长度的字符串是否为回文字符串。提示：由于要求字符串长度任意，所以用单链表存储字符串，即判断一个单链表是否为回文链表。

2) 分析

我们需要设计一个程序来判断一个单链表存储的字符串是否为回文字符串。回文字符串是指正读和反读都相同的字符串，例如“abba”和“abcba”。
总流程图 (3-19):

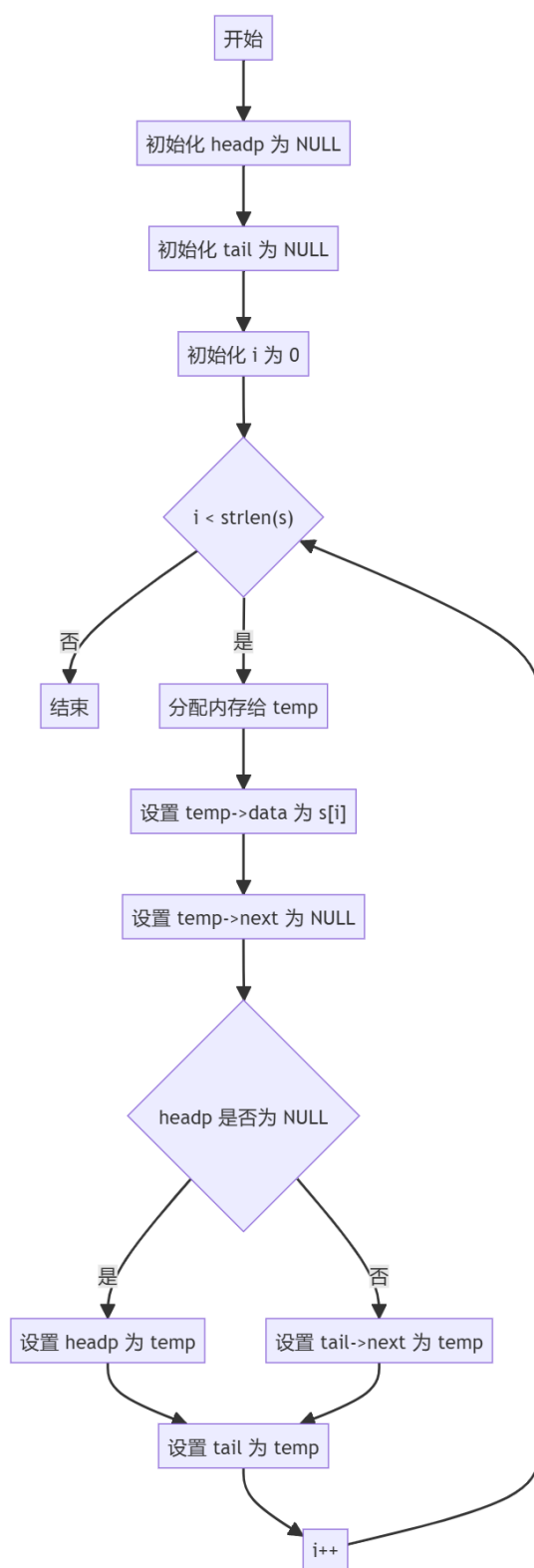


图 3-19 总流程图

思路：

选用单链表来存储字符串是因为题目要求能处理任意长度的字符串，单链表具有动态分配内存的特点，可以根据字符串的实际长度灵活地创建相应数量的节点来存储字符，避免了像数组那样需要预先指定固定大小可能导致空间浪费或者不够用的问题。

流程图 (3-20):

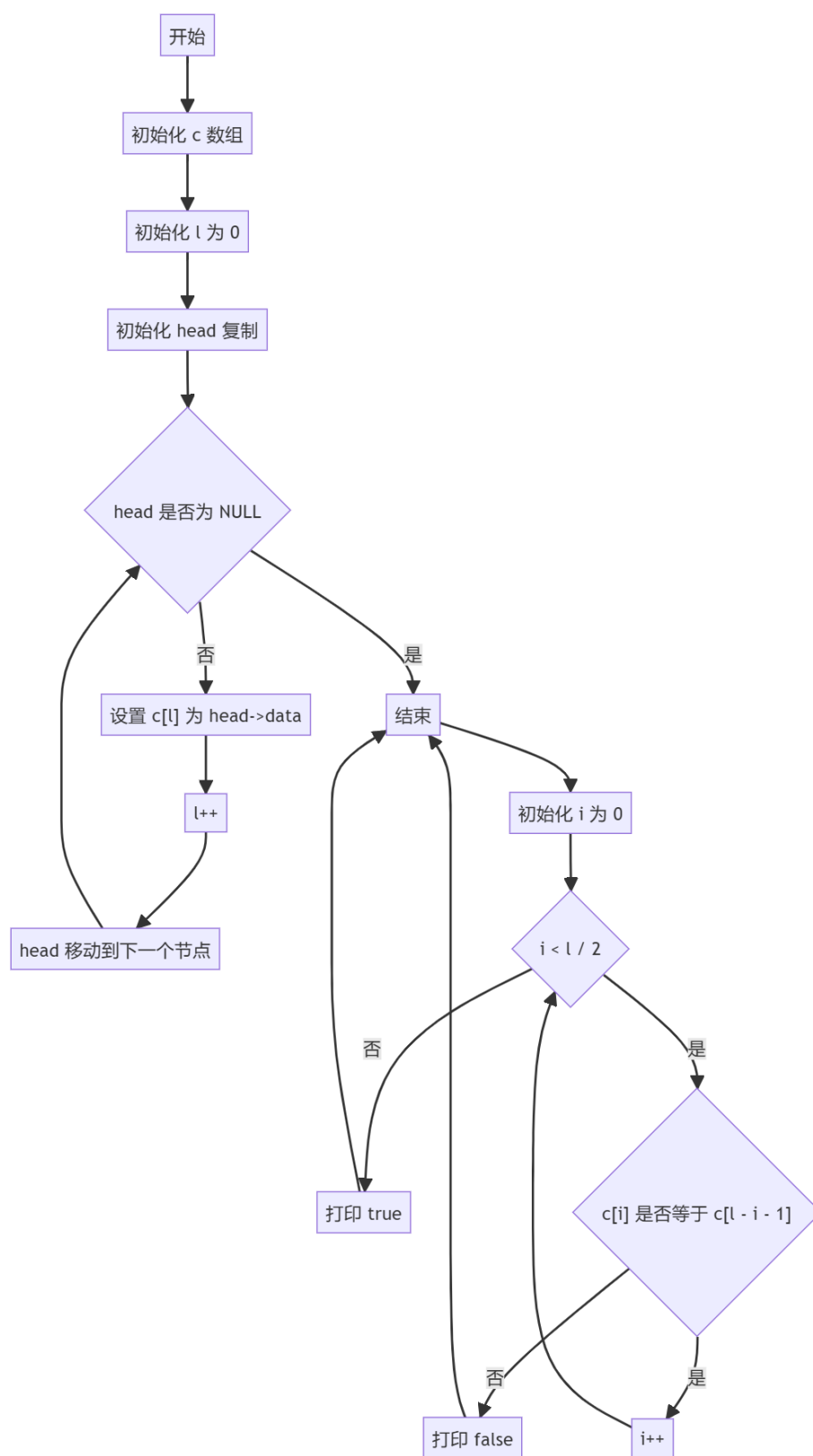


图 3-20 流程图

3) 相关代码

相关代码 (32):

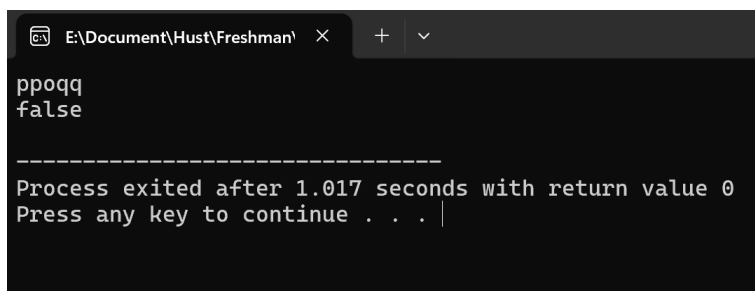
Listing 32 示例

```
1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <string.h>
4      typedef struct C_NODE {
5          char data;
6          struct C_NODE *next;
7      } C_NODE;
8      void createLinkList(C_NODE **headp, char s[])
9      {
10         C_NODE *tail;
11         *headp = NULL;
12         tail = NULL;
13         for(int i = 0; i < strlen(s); i++) {
14             C_NODE *temp = (C_NODE
15                             *)malloc(sizeof(C_NODE));
16             temp->data = s[i];
17             temp->next = NULL;
18             if (*headp == NULL) {
19                 *headp = temp;
20             } else {
21                 tail->next = temp;
22             }
23             tail = temp;
24         }
25     void judgePalindrome(C_NODE *head)
26     {
27         char c[1000];
28         int l = 0;
29         while(head != NULL) {
30             c[l++] = head->data;
31             head = head->next;
32         }
33         for (int i = 0; i < l / 2; i++) {
34             if (c[i] != c[l - i - 1]) {
35                 printf("false\n");
```

```
36         return;
37     }
38 }
39 printf("true\n");
40 }
41 int main() {
42     C_NODE *head = NULL;
43     char inputString[1000];
44     fgets(inputString, sizeof(inputString), stdin);
45     size_t len = strlen(inputString);
46     if (len > 0 && inputString[len - 1] == '\n') {
47         inputString[len - 1] = '\0';
48     }
49     createLinkList(&head, inputString);
50     judgePalindrome(head);
51     C_NODE *current = head;
52     while (current != NULL) {
53         C_NODE *next = current->next;
54         free(current);
55         current = next;
56     }
57     return 0;
58 }
```

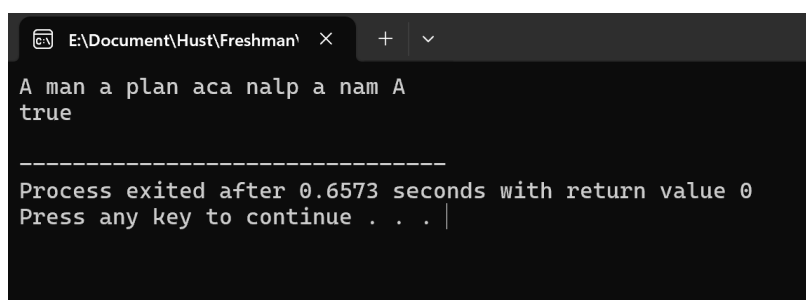
4) 执行结果

执行结果 (3-213-22):



```
E:\Document\Hust\Freshman' X + v
ppoqq
false
-----
Process exited after 1.017 seconds with return value 0
Press any key to continue . . . |
```

图 3-21 执行示例



```
E:\Document\Hust\Freshman\ > A man a plan aca nalp a nam A
true

-----
Process exited after 0.6573 seconds with return value 0
Press any key to continue . . . |
```

图 3-22 执行示例

可见通过上述的分析兴代码实现，构建了这个能识别回文的程序。

3.3.6 逆波兰表达式

1) 目标内容

利用值栈对逆波兰表达式进行求值。逆波兰表达式从键盘输入，其中的运算符仅包含加、减、乘、除 4 种运算，表达式中的数都是十进制数，用换行符结束输入。由于逆波兰表达式的长度不限，所以值栈要用后进先出链表实现。

2) 分析

本题旨在解决对从键盘输入的逆波兰表达式进行求值的问题。由于逆波兰表达式长度不限，所以采用后进先出链表来实现值栈，以适应不同长度表达式的计算需求，并且限定表达式中的运算符为加、减、乘、除四种基本运算，操作数为十进制数，通过栈操作和运算逻辑来得出最终的表达式求值结果。总流程图(3-23):

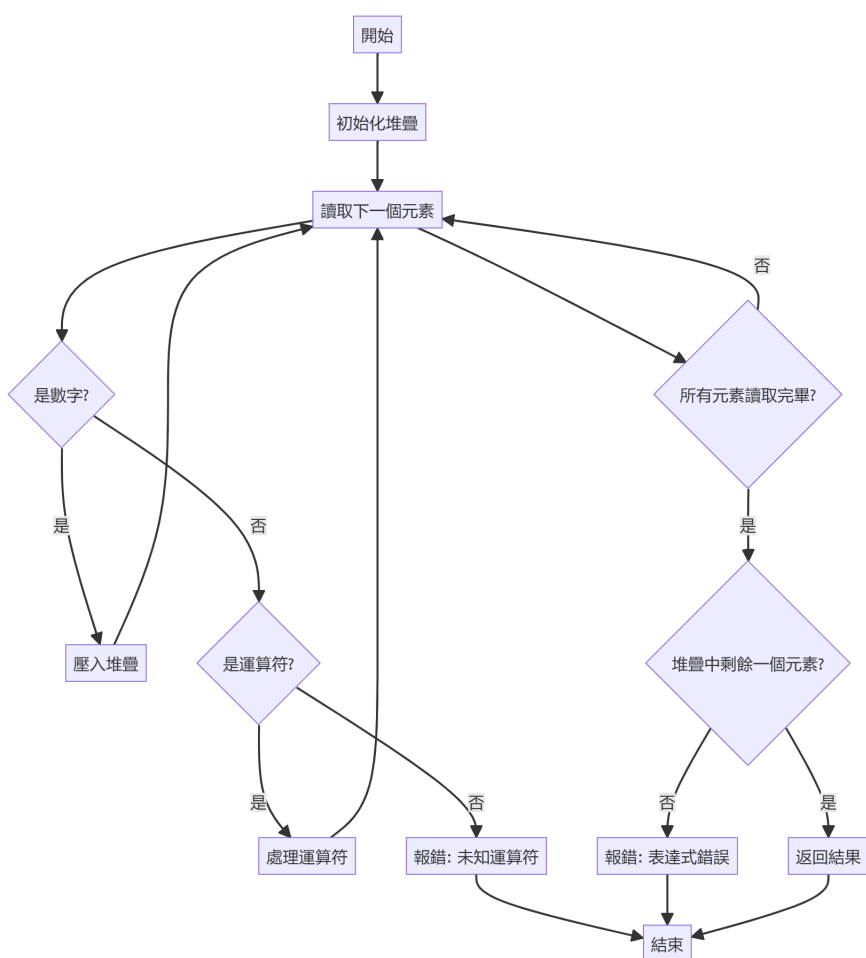


图 3-23 总流程图

思路：

本程序基于逆波兰表达式求值特点，选用后进先出链表实现值栈来构建求值程序。因逆波兰表达式计算顺序契合栈的后进先出特性且长度不限，链表可动态分配内存适应需求，故以链表实现栈。在 void 函数中，初始化栈顶指针为 NULL，按输入顺序处理元素，操作数入栈，运算符则弹出操作数计算后将结果入栈，处理完所有元素后根据栈状态确定求值结果并处理错误情况，最后在 main 函数中调用 void 函数求值并输出结果。相关流程图 (3-24)：

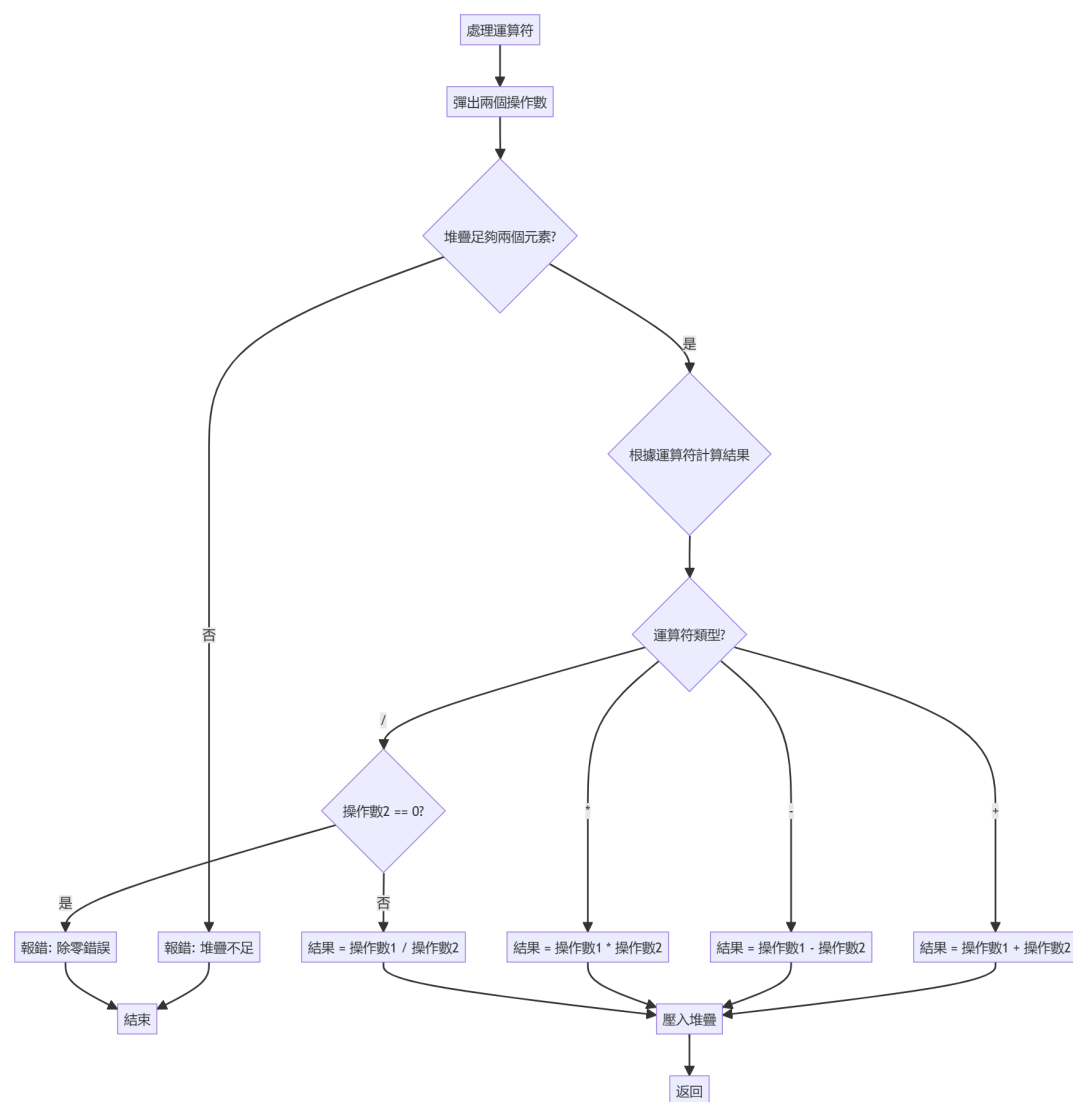


图 3-24 流程图

3) 相关代码

相关代码 (33):

Listing 33 示例

```

1      #include <stdio.h>
2      #include <stdlib.h>
3      #include <ctype.h>
4      #include <string.h>
5      typedef struct StackNode {
6          float value;
7          struct StackNode *next;
8      } StackNode;
    
```

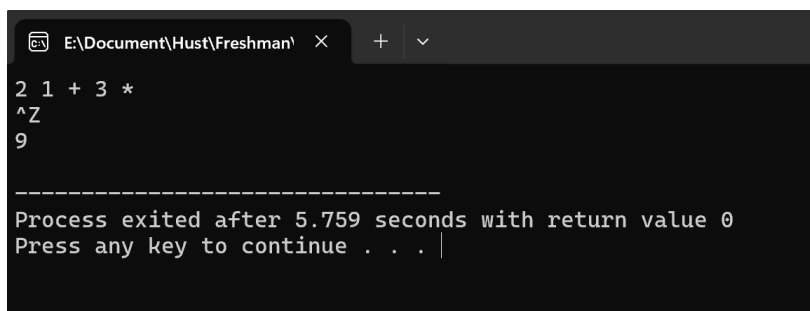
```
9         StackNode *createNode(float value) {
10             StackNode *newNode = (StackNode
11                                     *)malloc(sizeof(StackNode));
12             if (newNode == NULL) {
13                 fprintf(stderr, "内存分配失败\n");
14                 exit(1);
15             }
16             newNode->value = value;
17             newNode->next = NULL;
18             return newNode;
19         }
20         void push(StackNode **top, float value) {
21             StackNode *newNode = createNode(value);
22             newNode->next = *top;
23             *top = newNode;
24         }
25         float pop(StackNode **top) {
26             if (*top == NULL) {
27                 fprintf(stderr, "栈为空\n");
28                 exit(1);
29             }
30             StackNode *temp = *top;
31             float value = temp->value;
32             *top = (*top)->next;
33             free(temp);
34             return value;
35         }
36         int isEmpty(StackNode *top) {
37             return top == NULL;
38         }
39         float evaluateRPN() {
40             StackNode *top = NULL;
41             char input[100];
42             float operand1, operand2, result;
43             while (scanf("%s", input) != EOF) {
44                 if (isdigit(input[0]) || (input[0] == '-' &&
45                                         isdigit(input[1]))) {
46                     push(&top, atof(input));
47                 } else {
```

```
48         switch (input[0]) {
49             case '+':
50                 result = operand1 + operand2;
51                 break;
52             case '-':
53                 result = operand1 - operand2;
54                 break;
55             case '*':
56                 result = operand1 * operand2;
57                 break;
58             case '/':
59                 if (operand2 == 0) {
60                     fprintf(stderr, "除零错误\n");
61                     exit(1);
62                 }
63                 result = (int)(operand1 / operand2);
64                 break;
65             default:
66                 fprintf(stderr, "未知运算符: %c\n",
67                     input[0]);
68                 exit(1);
69             }
70         push(&top, result);
71     }
72     if (!isEmpty(top)) {
73         result = pop(&top);
74         if (isEmpty(top)) {
75             return result;
76         } else {
77             fprintf(stderr, "表达式错误\n");
78             exit(1);
79         }
80     } else {
81         fprintf(stderr, "表达式错误\n");
82         exit(1);
83     }
84 }
85 int main() {
86     float result = evaluateRPN();
87     printf("%.0f\n", result);
```

```
88         return 0;  
89     }
```

4) 执行结果

执行结果 (3-25):



```
E:\Document\Hust\Freshman\ > 2 1 + 3 *  
^Z  
9  
  
-----  
Process exited after 5.759 seconds with return value 0  
Press any key to continue . . . |
```

图 3-25 执行示例

可以见到，这也是成功读取文本并输出逆波兰表达式的结果。

3.4 实验七小结

在完成第七章有关结构的 C 语言实验题后，我对编程以及结构这一重要概念有了更为深刻和全面的理解，以下是我在这个过程中的一些心得体会与思维方式的总结。

3.4.1 对结构概念的深化理解

结构在 C 语言中提供了一种将不同类型的数据组合在一起的有效方式，形成一个逻辑上紧密相关的整体。它打破了基本数据类型只能单独表示单一信息的局限，使得我们能够以更贴近现实世界对象或问题域的方式来组织和处理数据。就比如，在处理学生成绩信息时，不再把数据信息一个个定义，而是通过定义一个 `Student` 结构，将这些相关的数据成员封装在一个统一的框架内。这不仅提高了代码的逻辑性和可读性，也使得数据的管理和操作更加方便和直观。我们可以一次性地传递整个学生信息结构体，而不是分别传递多个分散的数据，减少了函数参数的复杂性和出错的可能性。

从数据抽象的角度来看，结构是一种对现实世界实体进行抽象描述的有力工具。它隐藏了数据成员的具体实现细节，只对外展示与该结构相关的操作接口，符合面向对象编程中数据抽象和封装的初步思想。这种抽象能力有助于我们在大型

项目中构建复杂的系统，能够更好地组织和理解代码，提高代码的可维护性和可扩展性。

3.4.2 结构在编程实践中的优势

结构的使用可以促进代码的模块化设计。以一个班级成绩单管理系统为例，我们可以定义一个 `Student` 结构来表示学生信息，然后围绕这个结构设计一系列函数，如创建学生节点函数、添加学生信息到链表函数、打印学生信息函数，以及修改学生成绩函数等。每个函数专注于对 `Student` 结构数据的特定操作，使得整个程序的功能模块划分清晰，各模块之间职责明确，易于理解、维护和扩展。如果后续需要增加新的功能，比如按照成绩排序学生信息，我们可以基于已有的 `Student` 结构和相关函数进行拓展，而不会对整个程序架构造成较大的冲击。在处理具有内在关联性的数据时，结构能够显著提高处理效率。由于结构将相关数据整合在一起，在进行数据传递和操作时，减少了数据的拆分和重组过程。比如在计算学生的平均成绩时，直接访问 `Student` 结构中的各科成绩成员并进行计算，比分别传递各科成绩参数要简洁高效得多。而且，在涉及到多个函数对同一组相关数据进行操作时，结构作为参数传递能够保证数据的完整性和一致性，避免了因数据传递错误或不一致导致的程序错误。

3.4.3 编程思维方式的转变

以往在编写简单程序时，可能更多地关注单个数据元素的处理，如一个变量的赋值、运算等。而在接触和深入使用结构后，思维方式逐渐转变为从整体的数据结构出发来设计程序。在解决问题之前，首先考虑如何合理地定义数据结构来描述问题中的对象或实体，然后再基于这个数据结构规划相应的算法和操作函数。这种思维方式的转变使得编程不再是简单的代码堆砌，而是更具系统性和规划性的工程设计。例如，在设计一个图形绘制程序时，不再是孤立地考虑每个点的坐标绘制，而是先定义一个 `Shape` 结构，其中包含图形的类型、顶点坐标数组、颜色等相关信息，然后根据不同的图形类型（如圆形、矩形、三角形等）在相应的绘制函数中对 `Shape` 结构进行处理，这样的设计使得程序结构更加清晰，易于扩展和维护。

虽然 C 语言并非纯粹的面向对象编程语言，但结构的使用在一定程度上培养了面向对象的编程思维。通过将数据和对数据的操作封装在结构和相关函数中，初

步实现了数据的隐藏和封装，以及基于对象（结构实例）的操作设计。这让我在思考问题和编写代码时，开始注重对象的属性（数据成员）和行为（函数操作）的定义和设计，并且考虑如何通过对象之间的交互（函数调用和数据传递）来实现整个系统的功能。这种思维方式的转变为进一步学习和理解面向对象编程语言打下了坚实的基础，使得在接触更高级的面向对象编程概念时能够更快地适应和掌握。

3.4.4 遇到的问题与解决方案总结

在实验过程中，也遇到了不少问题，这些问题的解决过程进一步加深了我对结构编程的理解。

（一）结构体成员的访问与指针操作

在涉及到结构体指针的操作时，容易出现错误的指针引用和成员访问方式。例如，在通过链表操作学生信息结构体时，忘记对指针进行正确的解引用或者错误地使用 `->` 和 `.` 运算符来访问结构体成员。解决这个问题关键在于深入理解结构体指针的本质，明确指针变量所指向的是结构体的地址，通过 `->` 运算符可以访问指针所指向结构体的成员，而对于普通结构体变量则使用 `.` 运算符。同时，在进行复杂的链表操作时，仔细绘制链表结构示意图，标注每个指针的指向和结构体成员的位置，有助于清晰地理解代码逻辑，避免指针错误。

（二）结构与函数参数传递

在将结构体作为函数参数传递时，需要考虑传递方式对程序性能和数据安全性的影响。如果直接传递结构体变量，会进行整个结构体的拷贝，在结构体较大时可能会导致性能下降。而传递结构体指针虽然可以避免大量的数据拷贝，但需要注意指针的有效性和安全性，防止出现空指针引用或野指针问题。在实际编程中，根据具体情况权衡选择合适的传递方式，如果函数内部不需要修改结构体的内容，可以考虑将结构体声明为 `const` 类型并传递其指针，既保证了数据的安全性，又能提高程序性能。通过对这些问题的解决，我认识到在 C 语言结构编程中，细节决定成败。需要深入理解语言的特性和机制，仔细思考每一个代码片的逻辑和影响，才能编写出正确、高效、可维护的程序。

总之，完成第七章结构相关的 C 语言实验题是一次宝贵的学习经历，使我在编程思维、数据结构理解以及问题解决能力等方面都得到了显着的提升。结构作为 C 语言中重要的编程工具，为我揭示了编程代码的新境界，意识到自己的能力是

多少渺小，还需要学习的地方数之不尽，因此自己还需学习，不断求真。

4 總結心得

在 C 语言的学习之路上，从基本数据类型起始，逐步深入到条件运算符、新变量类型、数组、指针与结构的世界，这一过程犹如一场知识与思维的深度锤炼。基本数据类型如同大厦之基，`int`、`float`、`double` 与 `char` 依据各自特性为数据处理提供精准支撑，合理选用能兼顾程序正确性与效率。条件运算符则赋予程序逻辑判断的灵魂，`if-else` 等依条件抉择执行路径，其结合顺序不容小觑，严谨对待才能确保程序逻辑无偏差。新变量类型如 `enum` 与 `typedef` 拓展了表达边界，`enum` 让常量表意清晰，`typedef` 增强代码可移植性与维护性。

数组作为有序数据容器，高效存储同类型元素，下标访问便捷，但定长特性需谨慎对待，动态内存分配技巧可补其不足。指针恰似内存操控的魔杖，可间接访问变量、灵活分配内存、高效传参及构建复杂结构，然其风险并存，内存泄漏等隐患要求使用时务必精准无误。结构像是定制的数据收纳盒，整合不同类型成员，让数据处理更具整体性与逻辑性，于函数交互及复杂数据构建中表现卓越。

总体而言，学习 C 语言这些关键部分，收获的不仅是知识与技能，更是一种全方位的编程思维塑造。从数据考虑到逻辑规划，从结构设计到内存管理，多维度综合分析成为习惯，为未来编程进阶筑牢根基，开启深入探索编程世界，也让人深刻领略到 C 语言作为编程基石的深邃魅力与强大力量。

5 参考文献

以下為參考文獻：

1. 卢萍，李开，王多强，甘早斌.C 语言程序设计典型题解与实验指导, 北京: 清华大学出版社,2019
2. 卢萍，李开，王多强，甘早斌.C 语言程序设计, 北京: 清华大学出版社,2019